

FunDiff: Diffusion models over function spaces for physics-informed generative modeling

Sifan Wang^{1,+}, Zehao Dou^{2,+}, Siming Shan², Tong-Rui Liu³, and Lu Lu^{2,4,*}

¹Institute for Foundations of Data Science, Yale University, New Haven, CT 06511, USA

²Department of Statistics and Data Science, Yale University, New Haven, CT 06511, USA

³Laboratoire Navier, École nationale des ponts et chaussées, Institut Polytechnique de Paris, University Gustave Eiffel, CNRS UMR 8205, Champs-sur-Marne, 77420, France

⁴Department of Chemical and Environmental Engineering, Yale University, New Haven, CT 06511, USA

*Corresponding author. Email: lu.lu@yale.edu

⁺These authors contributed equally to this work.

ABSTRACT

Recent advances in generative modeling—particularly diffusion models and flow matching—have achieved remarkable success in synthesizing discrete data such as images and videos. However, adapting these models to physical applications remains challenging, as the quantities of interest are continuous functions governed by complex physical laws. To address this, we introduce **FunDiff**, an efficient and robust framework for generative modeling in function spaces. FunDiff combines a latent diffusion process with a function autoencoder architecture to handle input functions with varying discretizations, generates continuous functions that can be evaluated at arbitrary locations, and seamlessly incorporate physical priors. These priors are enforced through architectural constraints or physics-informed loss functions, ensuring that generated samples satisfy fundamental physical laws. We theoretically establish minimax optimality guarantees for density estimation in function spaces, demonstrating that diffusion-based estimators achieve optimal convergence rates under suitable regularity conditions. We further demonstrate the practical effectiveness of FunDiff across diverse applications in fluid dynamics and solid mechanics. Empirical results indicate that our method can generate physically consistent samples with high fidelity to the target distribution, and exhibit robustness to noisy and low-resolution data.

Introduction

Generative modeling, a cornerstone of modern machine learning, aims to learn and sampling complex data distributions $P(x)$ by constructing models that can generate new instances consistent with the learned distribution^{1,2}. Among recent advances in this domain, diffusion models and flow matching have emerged as powerful frameworks that achieve state-of-the-art performance. Diffusion models^{3,4} learn to reverse a forward noising process by gradually denoising data through a Markov chain, which iteratively transforms random noise into high-quality samples. In parallel, flow matching^{5,6} provides an alternative approach by learning continuous normalizing flows via optimal transport theory, directly modeling probability paths that transform a tractable prior distribution into the target data distribution.

Diffusion models have shown remarkable success across a wide range of applications, including high-resolution image synthesis^{7–9}, audio generation¹⁰, protein structure prediction¹¹, and molecular design¹². Despite these advances, the focus has largely remained on discrete data types, such as images, text, and molecular graphs, whereas many scientific domains, especially in physics, demand the generation of continuous function data, where each sample represents a continuous function (e.g., a velocity or temperature field) defined over spatial and temporal domains. For instance, generating velocity fields in turbulent fluid dynamics, simulating deformation patterns in complex materials under external loadings, and forecasting temperature variations in climate models all require handling functions defined over continuous spatial and temporal domains. These problems inherently involve infinite-dimensional function spaces, a challenge that recent advances in operator learning frameworks have begun to address effectively^{13–19}. This presents unique challenges:

How can we extend generative modeling to generate continuous functions that can be evaluated at arbitrary points, while preserving important structural or physical properties inherent in scientific problems?

Prior work in computer vision and graphics has extensively explored the concept of function-space generation through implicit neural representations and generative models, such as DeepSDF²⁰, NeRF²¹, From Data to Functa²², and SIREN²³, as well as their diffusion-based extensions including 3DShape2VecSet²⁴, Shap-E²⁵, and HyperDiffusion²⁶. These methods treat signals such as shapes, radiance fields, or signed distance functions as continuous functions parameterized by neural

networks, establishing an important foundation for generative modeling in continuous domains. Furthermore, recent works have attempted to extend the framework of generative models to function space, either by adapting existing algorithms to operate on discretized function data in $\mathbb{R}^n$ or by transforming input functions into a finite-dimensional latent space^{22,27–31}. Dupont et al.³² pioneered this direction by treating data points as continuous functions and learning distributions over them using adversarial training. Multiple theoretical frameworks have emerged for function diffusion models, including Gaussian measures on Hilbert spaces³³, dimension-independent bounds for infinite-dimensional models³⁴, generalized diffusion processes in function spaces³⁵, and denoising diffusion operators with rigorous Gaussian processes and Langevin dynamics³⁶. Although these studies have demonstrated promising results for fixed discretizations, they generally struggle to scale to higher resolutions or to handle variable discretizations consistently, as those encountered in scientific computing, partial differential equation (PDE) modeling, or operator learning^{14,37–40}.

In parallel, diffusion models have emerged as powerful tools for tackling inverse problems and modeling dynamical systems governed by PDEs. Recent works have developed methods for reconstructing spatiotemporal fields from sparse observations and partial supervision^{41–43}. To address resolution limitations, several approaches have integrated neural operators with diffusion models^{44–46}, demonstrating improved performance in turbulence modeling and capture of multiscale phenomena. Furthermore, various strategies have attempted to incorporate physical priors directly into diffusion processes^{42,47–52}. For instance, DiffusionPDE⁴² introduces physics-based guidance during the posterior sampling stage, whereas PIDM⁴⁹ embeds PDE residuals directly into the training objective. However, enforcing PDE residuals is highly sensitive to solution accuracy—small numerical errors can destabilize both training and inference. Moreover, physical constraints are inherently defined in continuous space, but these diffusion models operate on discretized representations. This mismatch introduces discretization artifacts, often leading to inaccurate or ill-posed PDE residuals, especially in inverse problems where key state variables are only partially observed. These challenges underscore a fundamental gap between current data-driven generative models and the objective of producing physically consistent, continuous functions.

To address these challenges, we propose FunDiff, a novel framework for physics-informed generative modeling over function spaces. Our contributions are as follows. We introduce a unified architecture that combines latent diffusion processes with function autoencoders, enabling the high-quality data generation in a function-space representation. This design supports evaluation at arbitrary spatial locations while seamlessly incorporating physical priors through the function space representation. We establish theoretical guarantees for our approach by proving that diffusion-based models attain minimax-optimal rates for density estimation in infinite-dimensional spaces under mild smoothness assumptions. We demonstrate the practical effectiveness of FunDiff across diverse applications in fluid dynamics and solid mechanics. Our experiments show that the method generates physically consistent samples with high fidelity to target distributions while maintaining robustness to noisy and low-resolution input data. Taken together, our framework bridges the gap between theoretical guarantees in function spaces and practical neural network algorithms, establishing a principled approach to physics-informed generative modeling.

Results

In this section, we present theoretical results on generative models in function spaces, which establish the mathematical foundation of our work. Building on this theory, we introduce FunDiff, a practical framework for generating functions while incorporating physical priors. Then we demonstrate the effectiveness of our proposed FunDiff framework through a series of experiments on diverse physical systems. Specifically, we present four case studies with increasing complexity: (1) the Burgers’ equation, showcasing the incorporation of PDE constraints; (2) Kolmogorov flow, highlighting divergence-free field generation; (3) linear elasticity, illustrating symmetry preservation; and (4) turbulent mass transfer, demonstrating conditional generation capabilities. For notations, interested readers are encouraged to refer to the Appendix S1. It is important to emphasize that in all examples, FunDiff does not act as a neural PDE solver; rather, it learns to generate physically consistent continuous functions that approximate the distributions of solutions to selected PDE-governed systems when trained on such data.

Generative models over function spaces

In this section, we introduce a framework for generative modeling in function spaces. Given the inherently challenges of handling directly with infinitely-dimensional spaces, we begin by reducing the problem to a finite-dimensional setting, with preliminaries in Appendix S2. By embedding functions into a finite-dimensional vector space, the task of generating new functions can be reformulated as a well-studied vector generation task.

To formalize this idea, let us begin with a vector space generative oracle $\mathcal{O}(d, n)$, which, given n i.i.d. samples $\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^n \in \mathbb{R}^d$ from an underlying distribution $\mu \in \mathcal{P}(\mathbb{R}^d)$, produces an estimate $\hat{\mu} \in \mathcal{P}(\mathbb{R}^d)$ of μ . This oracle is assumed to achieve the minimax statistical estimation rate

$$\mathbb{E}[W_1(\hat{\mu}, p)] \lesssim \varepsilon(d, n),$$

where the expectation is taken over the randomness of the samples, and $\varepsilon(d, n)$ aligns with the known minimax estimation rates^{53,54}. For the continuous density class $\mathcal{C}(\mathbb{R}^d) := \{\mu : \mathbb{R}^d \rightarrow [0, \infty) : \mu \text{ is continuous}\}$, we have a minimax density estimation rate

$$\sup_{\hat{\mu}} \inf_{\mu \in \mathcal{C}(\mathbb{R}^d)} \mathbb{E}[W_1(\hat{\mu}, \mu)] \asymp n^{-1/d}$$

for $d > 2$, where $\hat{\mu}$ is generated by diffusion models in this paper. It is a special case for the $n^{-\frac{\alpha+1}{2\alpha+d}}$ minimax rate when the smoothness $\alpha = 0$. Therefore, for the entire continuous density class $\mathcal{C}(\mathbb{R}^d)$, we have $\varepsilon(d, n) = n^{-1/d}$.

Building upon this oracle, we seek to develop a generative modeling method that operates over a space of functions. Suppose that we have n functions $f^{(1)}, f^{(2)}, \dots, f^{(n)} : \mathbb{R}^d \rightarrow \mathbb{R}$, drawn i.i.d. from an unknown distribution $P \in \mathcal{P}(\mathcal{F})$, where $\mathcal{F}$ is a subset of the continuous function space $\mathcal{C}(\mathbb{R}^d)$. Our goal is to devise a theoretically sound methodology for generating new function samples from P , given only these observed samples.

To achieve this, we introduce a function autoencoder (FAE) framework. We define an encoder $\mathcal{E} : \mathcal{F} \rightarrow \mathbb{R}^D$, which maps a function to a finite-dimensional vector representation, and a decoder $\mathcal{D} : \mathbb{R}^D \rightarrow \mathcal{F}$, which maps these vectors back to functions. Applying $\mathcal{E}$ to the given functions $f^{(1)}, f^{(2)}, \dots, f^{(n)}$ yields vector samples $\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(n)} \in \mathbb{R}^D$. We then use the vector-space generative oracle $\mathcal{O}(D, n)$ on these vectors to obtain an estimated distribution $\hat{p} \in \mathcal{P}(\mathbb{R}^D)$. Finally, we can sample new vectors $\theta \in \mathbb{R}^D$ from $\hat{p}$ and apply $\mathcal{D}$ to generate new functions $f = \mathcal{D}(\theta) \in \mathcal{F}$, the distribution of these newly generated functions is indicated by $\hat{P}$ and serves as an estimation of P . This construction effectively leverages the theoretical foundations of vector-space generative modeling and extends them to the realm of infinite-dimensional function spaces.

In order to measure the distribution loss, we apply an analogy of the Wasserstein distance $W_1(P, \hat{P})$. Notice that for distributions over the function space, the Wasserstein distance is defined similarly as

$$W_1(P, \hat{P}) := \sup_{\text{Lip}(\mathcal{L}) \leq 1} \left[\mathbb{E}_{f \sim P} \mathcal{L}(f) - \mathbb{E}_{f \sim \hat{P}} \mathcal{L}(f) \right],$$

where $\mathcal{L}$ is a functional and its Lipschitz constant means:

$$\text{Lip}(\mathcal{L}) = \sup_{f, g} \frac{\mathcal{L}(f) - \mathcal{L}(g)}{\|f - g\|_2}.$$

In the next part, we propose an upper bound of $W_1(P, \hat{P})$ and explain all its factors.

Density estimation loss decomposition

Suppose the vector representations $\{\theta^{(k)} = \mathcal{E}(f^{(k)}) \mid k \in [n]\}$ are drawn from a distribution $p = E_{\#}P$, which stands for the pushforward of P under the encoder $\mathcal{E}$. Applying the vector-space generative oracle $\mathcal{O}(D, n)$ to these samples yields an estimated distribution $\hat{p} \in \mathcal{P}(\mathbb{R}^D)$. We then feed $\hat{p}$ into the decoder $\mathcal{D}$, producing $G_{\#}\hat{p}$, an estimate of the original distribution P in the function space. For the estimation loss, we have the bound

$$W_1(P, \hat{P}) \leq \mathbb{E}_{f \sim P} \|f - \mathcal{D} \circ \mathcal{E}(f)\|_2 + \text{Lip}(\mathcal{D}) \cdot W_1(p, \hat{p}). \quad (1)$$

Here, the first term $\mathbb{E}_{f \sim P} \|f - \mathcal{D} \circ \mathcal{E}(f)\|_2$ represents the generalization error for the autoencoder $\mathcal{D} \circ \mathcal{E} : \mathcal{F} \rightarrow \mathcal{F}$. The second term $\text{Lip}(\mathcal{D})$ stands for the Lipschitz constant of the decoder $\mathcal{D}$, defined by the inequality

$$\|\mathcal{D}(\theta) - \mathcal{D}(\theta')\|_2 \leq \text{Lip}(\mathcal{D}) \cdot \|\theta - \theta'\|_2.$$

This quantity measures the robustness of $\mathcal{D}$ with respect to perturbations in the input vector space. The third term, $W_1(p, \hat{p})$, reflects the accuracy of the density estimation for vector space, highlighting the performance of the oracle $\mathcal{O}(D, n)$ in estimating the true distribution p . The dimension of the feature space D must be chosen judiciously to strike a balance between ensuring accurate autoencoder reconstruction and mitigating the effects of the curse of dimensionality for the vector-space generative oracle. The proof of (1) can be seen in Appendix S3.1. Next, we propose some simple examples to help us understand the theoretical framework.

Illustrative example 1: Reproducing kernel Hilbert space (RKHS). For a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$, we denote its eigenvalues as $\mu_1 \geq \mu_2 \geq \dots$, and their corresponding eigenfunctions as $\psi_1, \psi_2, \dots$. For any function $f \in \mathcal{H}$, its eigen-decomposition has the form $f = \sum_{i=1}^{\infty} \theta_i \sqrt{\mu_i} \psi_i$ and its RKHS norm is defined as $\|f\|_{\mathcal{H}}^2 = \sum_{i=1}^{\infty} \theta_i^2$. Define the RKHS unit ball as $\mathcal{B} := \{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq 1\}$.

Given n sample functions $f^{(1)}, f^{(2)}, \dots, f^{(n)} \sim P \in \mathcal{P}(\mathcal{B})$, assume their eigen-decomposition is

$$f^{(k)} = \sum_{i=1}^{\infty} \theta_i^{(k)} \cdot \sqrt{\mu_i} \psi_i, \quad (k = 1, 2, \dots, n).$$

We define the encoder $\mathcal{E} : \mathcal{B} \rightarrow \mathbb{R}^D$ as the truncation of D coefficients and the decoder $\mathcal{D} : \mathbb{R}^D \rightarrow \mathcal{B}$ as the linear combination:

$$\mathcal{E} : f \mapsto (\theta_1, \theta_2, \dots, \theta_D)^\top, \quad \mathcal{D} : (\theta_1, \theta_2, \dots, \theta_D)^\top \mapsto \sum_{i=1}^D \theta_i \cdot \sqrt{\mu_i} \psi_i.$$

Hence, the autoencoder $\mathcal{D} \circ \mathcal{E}$ effectively serves as a projection (truncation) onto the subspace spanned by the first D eigenfunctions. Next, in order to upper bound the density estimation loss $W_1(P, \hat{P})$, we introduce two different assumptions corresponding to the polynomial and exponential eigenvalue decay of the RKHS $\mathcal{H}$.

Assumption 1 (2β -polynomial decay). Assume that the RKHS $\mathcal{H}$ is embedded to a β -th order Sobolev space where $\beta > \frac{1}{2}$. It leads to the property that the eigenvalues with respect to the kernel function $K(\cdot, \cdot)$ have the property of 2β -polynomial decay as $\mu_i \asymp i^{-2\beta}$ for all $i \geq 1$. To be more specific, we assume that

$$c_{\mathcal{H}} \cdot i^{-2\beta} \leq \mu_i \leq C_{\mathcal{H}} \cdot i^{-2\beta}$$

holds for $\forall i \geq 1$, where $c_{\mathcal{H}}$ and $C_{\mathcal{H}}$ are constants.

Assumption 2 (γ -exponential decay). Assume that the eigenvalues with respect to the kernel function $K(\cdot, \cdot)$ have the property of γ -exponential decay as $\mu_i \asymp \exp(-C_1 \cdot i^\gamma)$ for all $i \geq 1$ where $C_1 > 0$ is a positive constant. To be more specific, we assume that

$$c_{\mathcal{H}} \cdot \exp(-C_1 \cdot i^\gamma) \leq \mu_i \leq C_{\mathcal{H}} \cdot \exp(-C_1 \cdot i^\gamma)$$

holds for $\forall i \geq 1$, where $c_{\mathcal{H}}$ and $C_{\mathcal{H}}$ are constants.

Then, the reconstruction loss $\mathbb{E}_{f \sim P} \|f - \mathcal{D} \circ \mathcal{E}(f)\|_2$ represents the truncation loss under the RKHS eigen-decomposition, and the robustness $\text{Lip}(\mathcal{D})$ is directly related to the RKHS norm bound of $\mathcal{B}$.

Theorem 1 (Density estimation over RKHS space). Assume P is a distribution supported on the RKHS unit ball $\mathcal{B} := \{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq 1\}$. By using the generative oracle $\mathcal{G}(D, n)$ and the function autoencoder (FAE) framework introduced above, the density estimation loss over RKHS space can be bounded as follows:
Under the polynomial decay setting:

$$W_1(P, \hat{P}) \lesssim \left(\frac{\log \log n}{\log n} \right)^\beta.$$

Under the exponential decay setting:

$$W_1(P, \hat{P}) \lesssim \exp \left(-(C_1/2)^{\frac{1}{1+\gamma}} (\log n)^{\frac{\gamma}{1+\gamma}} \right),$$

after we choose an optimal latent dimension D .

Proof. The proof is provided in Appendix S3.2. □

Illustrative example 2: Barron space. First, we introduce the concept of Barron space⁵⁵. For function $f : \Omega := [0, 1]^d \rightarrow \mathbb{R}$, we aim to recover f using a fully connected two-layer neural network with ReLU activation:

$$f(\mathbf{x}; \theta) = \sum_{k=1}^m a_k \sigma(\mathbf{b}_k \cdot \mathbf{x} + c_k).$$

Here, $\sigma(\cdot)$ is the component-wise ReLU activation $\sigma(t) = \max(0, t)$, $\mathbf{b}_k \in \mathbb{R}^d$, $a_k, c_k \in \mathbb{R}$ for $k \in [m]$, and the whole parameter set $\theta = \{(a_k, \mathbf{b}_k, c_k)\}_{k=1}^m$ is to be learned, and m is the width of the network. As we can see, $\theta \in \mathbb{R}^{m(2+d)}$. In order to control the magnitude of the learned network, we use the following scale-invariant norm.

Definition 2 (Path norm⁵⁶). For a two-layer ReLU network, the path norm is defined as

$$\|\theta\|_P = \sum_{k=1}^m |a_k| \cdot (\|\mathbf{b}_k\|_1 + |c_k|).$$

Definition 3 (Spectral norm). Given $f \in L^2(\Omega)$, denote by $F \in L^2(\mathbb{R}^d)$ an extension of f to $\mathbb{R}^d$. Let $\hat{F}$ be the Fourier transform of F , then

$$f(\mathbf{x}) = \int_{\mathbb{R}^d} e^{i\langle \mathbf{x}, \omega \rangle} \hat{F}(\omega) d\omega, \quad \forall \mathbf{x} \in \Omega.$$

We define the spectral norm of f by

$$\gamma(f) = \inf_{F \in L^2(\mathbb{R}^d), F|_{\Omega} = f|_{\Omega}} \int_{\mathbb{R}^d} \|\omega\|_1^2 \cdot |\hat{F}(\omega)| d\omega.$$

Assumption 3. We consider all functions with a bounded value and a bounded spectral norm.

$$\mathcal{F}_s = L^2(\Omega) \cap \{f(\mathbf{x}) : \Omega \rightarrow \mathbb{R} \mid \gamma(f) \leq 1, \|f\|_{\infty} \leq 1\}.$$

We let the function class $\mathcal{F} = \mathcal{F}_s$, which makes the distribution P supported on Barron space $\mathcal{F}_s$.

In the following analysis, we mainly rely on the approximation property for two-layer ReLU networks over the Barron space $\mathcal{F}_s$, which are proposed in Refs.^{55,57,58}.

Lemma 4. For any $f \in \mathcal{F}_s$, there exists a two-layer network $f(\mathbf{x}; \tilde{\theta})$ of width m such that:

$$\|f(\mathbf{x}; \tilde{\theta}) - f(\mathbf{x})\|_{\infty}^2 \leq \frac{16\gamma^2(f)}{m} \leq \frac{16}{m}.$$

Furthermore, the parameter $\tilde{\theta}$ can be bounded as:

$$\|\mathbf{b}_k\|_1 = 1, |c_k| \leq 1, \text{ and } |a_k| \leq \frac{2\gamma(f)}{m} \leq \frac{2}{m} \quad \text{holds for } \forall k \in [m].$$

In the context of the FAE framework, we set our encoder as $\mathcal{E} : f \mapsto \tilde{\theta}$. If there exist multiple valid $\tilde{\theta}$ for $f \in \mathcal{F}_s$, we can choose any one of them as the encoder's output. As we see, the encoder $\mathcal{E}$ maps the function of the Barron space to a parameter set with bounded path norm. The decoder $\mathcal{D} : \theta \mapsto f(\mathbf{x}; \theta)$ is simply a parameter injection of a two-layer ReLU network. Hence, the autoencoder $\mathcal{D} \circ \mathcal{E}$ effectively serves as a two-layer ReLU network approximation. Next, we upper bound the density estimation loss $W_1(P, \hat{P})$.

Theorem 5 (Density estimation over Barron space). Assume P is a distribution supported on the Barron space $\mathcal{F}_s$. By using the generative oracle $\mathcal{O}(D, n)$ and the function autoencoder (FAE) framework introduced above, the density estimation loss over Barron space can be bounded as

$$W_1(P, \hat{P}) \lesssim \sqrt{\frac{d \log d}{\log n}}.$$

Proof. The proof is provided in Appendix S3.3. □

Now we have proved the statistical upper bound of density estimation over the RKHS ball and Barron space. To further show the optimality of such an upper bound, we provide a statistical lower bound of density estimation over the RKHS ball, which matches its upper bound with only a negligible $\mathcal{O}(\text{poly}(\log \log n))$ gap.

Theorem 6 (Minimax lower bound for Wasserstein estimation over an RKHS ball). Assume $\mathcal{H}$ is an RKHS following 2β -polynomial eigenvalue decay (Assumption 1)

$$c_{\mathcal{H}} \cdot i^{-2\beta} \leq \mu_i \leq C_{\mathcal{H}} \cdot i^{-2\beta}$$

and define the RKHS unit ball $\mathcal{B} := \{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq 1\}$. We observe n i.i.d. samples $X_1, X_2, \dots, X_n \sim P$, where the unknown distribution P is supported on the RKHS ball $\mathcal{B}$. Let the statistical minimax rate

$$\mathcal{R}_n := \inf_{\hat{P}_n} \sup_{P: \text{supp}(P) \subseteq \mathcal{B}} \mathbb{E} \left[W_1(P, \hat{P}_n) \right].$$

Then there exists a constant $K(\beta, c, C) > 0$, depending only on pure constants β, c, C in the eigenvalue decay assumption, such that for all sufficiently large n ,

$$\mathcal{R}_n \geq K(\beta, c, C) \cdot \left(\frac{1}{\log n} \right)^\beta.$$

Proof. The proof is provided in Appendix S3.4. □

Notice that its corresponding statistical upper rate is $\left(\frac{\log \log n}{\log n} \right)^\beta$ according to our results in Theorem 1. It means that our upper and lower bounds only have a $\mathcal{O}(\text{poly}(\log \log n))$ gap, which is negligible to our statistical rate.

FunDiff in practice

To realize the framework of function generative models outlined above, we propose FunDiff (Fig. 1), which consists of a novel FAE and a latent diffusion model to generate continuous functions. The FAE learns the mapping from discrete function representations to continuous ones, while also enabling the enforcement of prior physical constraints. The latent diffusion model is then trained directly on the latent manifold learned by the pretrained FAE encoder. Our generation process follows a simple two-step procedure: we first sample latent codes by integrating the reverse-time ODE from Gaussian noise to the target data distribution, and then transform these codes into continuous functions via the FAE decoder.

Physical prior integration. The FAE framework enables flexible integration of physical priors through multiple mechanisms of either soft or hard constraints. For general PDE constraints, we augment the loss with a residual term:

$$\mathcal{L}_{\text{PDE}} = |\mathcal{R}(\hat{f})|_2^2,$$

where $\mathcal{R}$ represents the PDE residual and $\hat{f}$ is the reconstructed function. The continuous representation of FAE enables automatic differentiation to compute all required derivatives, allowing constraint enforcement across the entire domain. It is worth noting that these physics-informed terms serve as regularizers rather than numerical solvers of the governing equations; they guide the learned FAE to produce output functions that are physically consistent.

Furthermore, some physical properties can be incorporated through architectural modifications. For periodic boundary conditions, one approach is to modify the coordinate embedding with Fourier bases of specific frequencies⁵⁹. When handling divergence-free fields, particularly important in fluid dynamics applications, the FAE can be configured to learn a stream function ψ , from which velocity field components are derived as $u = \frac{\partial \psi}{\partial y}$, $v = -\frac{\partial \psi}{\partial x}$, thereby ensuring divergence-free flow. Symmetry constraints can be incorporated when needed through modifications to the decoder’s final layer.

Related work. We emphasize that our approach fundamentally differs from existing physics-informed diffusion models^{42,47–52}, which impose physical constraints either by modifying the diffusion model’s training objective or by guiding the posterior sampling process. These methods operate on discretized outputs, where physical laws—typically enforced through PDE residuals—are subject to discretization errors. This not only leads to inaccurate constraint enforcement, but also introduces additional optimization challenges. Moreover, in inverse problems where key physical variables are unobserved, such residuals may be undefined at inference time, further limiting the applicability of these approaches. In contrast, our method enforces physical constraints within the FAE, which is fully decoupled from the training and inference processes of the latent diffusion model. Since the FAE provides a continuous neural representation, we can compute PDE residuals by automatic differentiation when needed. This design enables both exact and soft enforcement of constraints in continuous space, mitigating discretization-induced errors, and enabling more stable and physically consistent generation. Details on how to impose these constraints are provided in the following examples through comprehensive numerical experiments, demonstrating that our framework can generate diverse function samples while naturally preserving the underlying physical principles.

Learning 1D damped sinusoidal functions

To validate our theoretical analysis and demonstrate the training pipeline of the proposed framework, we first present a pedagogical example using a simple distribution of one-dimensional damped sinusoidal signals. The target functions are defined as

$$f(x) = A \exp(-\gamma x) \sin(\omega x) + b, \quad x \in [0, 1], \quad (2)$$

where the parameters are sampled from uniform distributions: amplitude $A \sim \mathcal{U}[0.5, 1]$, decay rate $\gamma \sim \mathcal{U}[2, 4]$, angular frequency $\omega \sim \mathcal{U}[6\pi, 8\pi]$, and vertical offset $b \sim \mathcal{U}[-0.5, 0.5]$.

We employ an FAE with a 6-layer encoder and a 2-layer decoder, each with a hidden dimension of 256 and 8 attention heads. The FAEs are trained on datasets of varying sizes, with function samples discretized at 128 uniform grid points. We

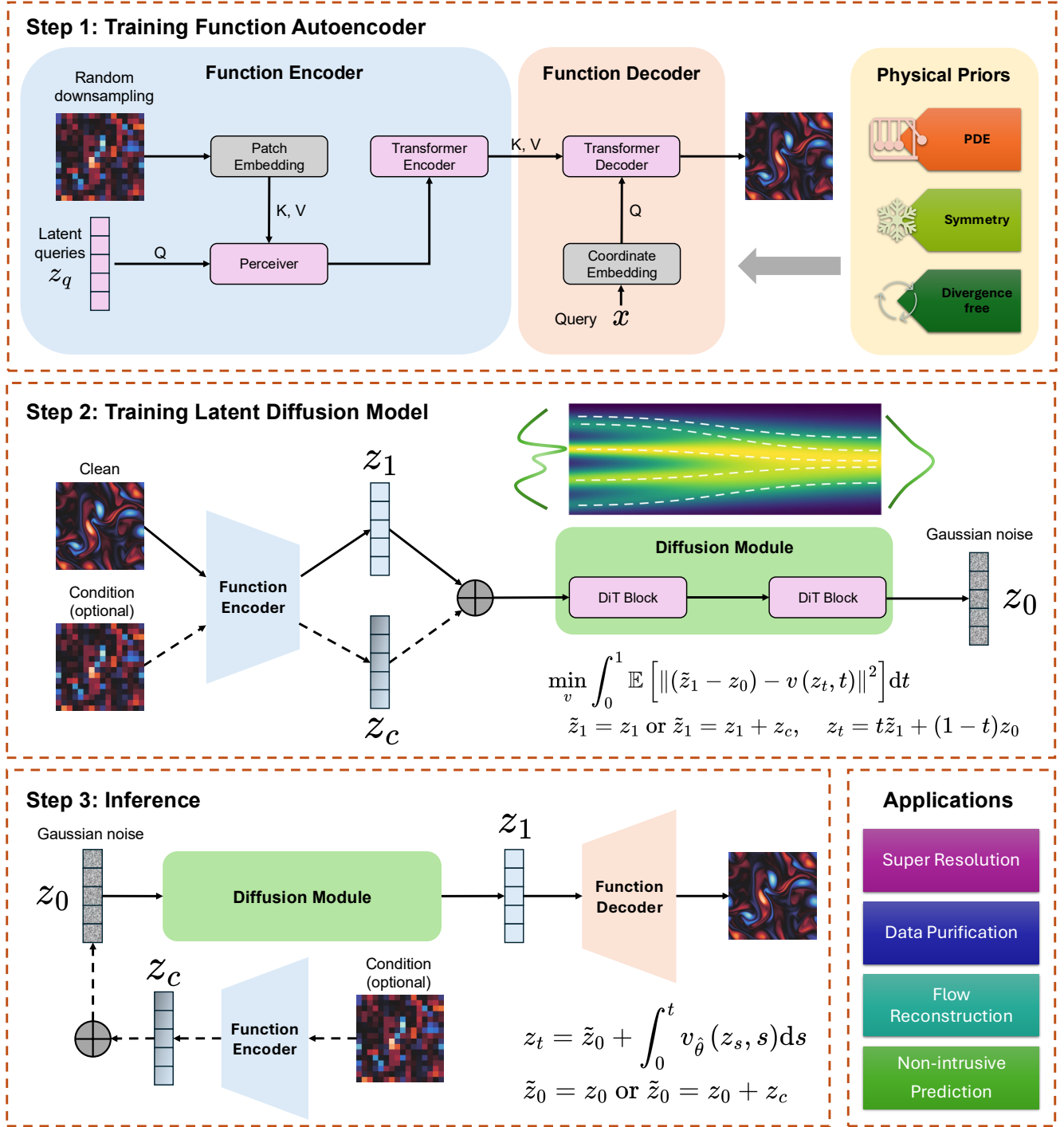


Figure 1. Overview of the function generative framework. The framework comprises three main steps. (1) We first train a function autoencoder (FAE) that maps discretized function data into a continuous latent space while preserving physical constraints. The encoder employs a Perceiver module to handle variable discretizations and project them into a unified latent representation. A Vision Transformer (ViT) processes the discretized inputs, and the decoder enables continuous function evaluation via cross-attention between query coordinates x and encoded features. When known physical priors are available, the decoder is modified accordingly. (2) Next, we train a Diffusion Transformer (DiT) in the learned latent space to generate diverse function samples. For conditional generation, partial observations are encoded using the pretrained transformer and incorporated into the diffusion model inputs. The diffusion model is trained using rectified flow. (3) During inference, latent codes are sampled by solving a reverse-time ODE from Gaussian noise, and then decoded into continuous functions via the decoder. The framework naturally supports various physical constraints through its continuous latent representation, enabling applications in super-resolution, flow reconstruction, data purification, and non-intrusive prediction.

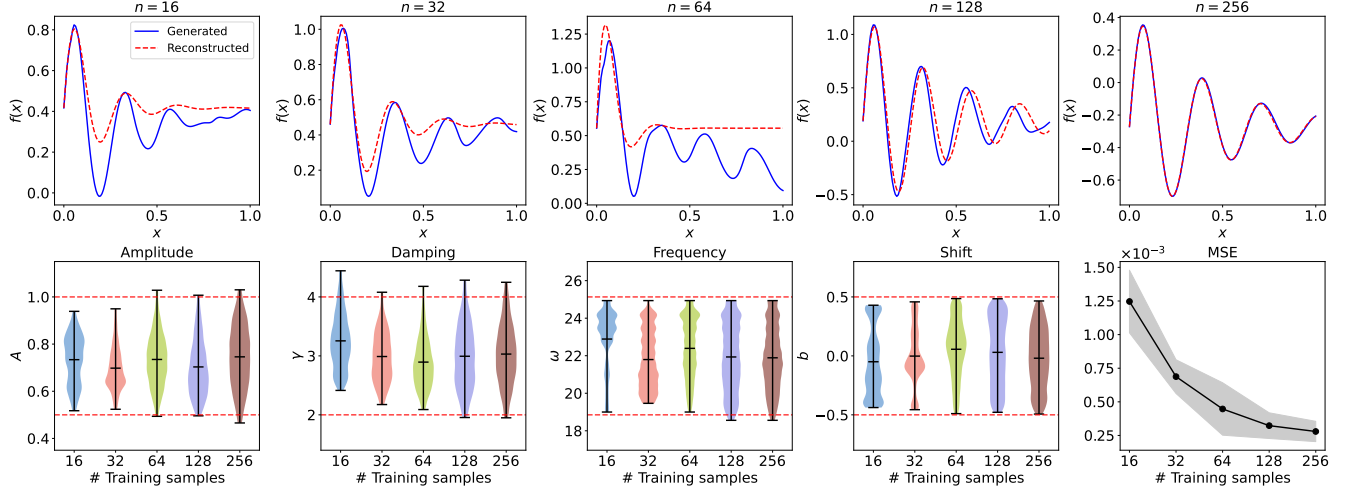


Figure 2. Damped sinusoidal functions. Analysis of generative quality across models trained with different numbers of samples ($n = 16, 32, 64, 128, 256$). **(Top)** Comparison between the worst-case generated samples (blue dots) and their reconstructed signals (red dashed lines) obtained through parameter estimation. **(Bottom)** The first four panels show the estimated parameter distributions for amplitude (A), damping coefficient (γ), angular frequency (ω), and vertical shift (b). Red dashed lines indicate the true parameter bounds from the underlying uniform distributions. The rightmost panel displays the MSE of the reconstructed signal. The results demonstrate that models trained with more samples generate functions that better align with the true distributions and achieve lower reconstruction error.

train the FAE by minimizing the loss using the Adam optimizer for 5×10^4 iterations. The learning rate follows a warmup and exponential decay schedule: it linearly increases from 0 to 10^{-3} over the first 1,000 steps, then exponentially decays every 1,000 steps with a decay rate of 0.9. Following the FAE training, we train the latent diffusion model by minimizing (9) using an identical learning rate schedule.

For evaluation, we generate 8,192 sample functions using our inference procedure with 100 denoising steps. We assess the quality of these generated samples through parameter estimation, leveraging the analytical form of damped sinusoidal functions. Specifically, for each generated sample, we perform a least-squares fitting using (2) to obtain the inferred parameters and reconstruct the corresponding signal. This approach enables a rigorous quantitative assessment of our model’s ability to capture the underlying function space through two metrics: the comparison between reconstructed and generated signals, and the analysis of inferred parameter distributions against the target uniform distributions.

We provide a comprehensive evaluation of our model’s generative performance across varying training dataset sizes (Fig. 2). The top row illustrates the worst-case generated samples alongside their reconstructions via parameter estimation. As the number of training samples increases, the quality of generation improves significantly. The bottom row analyzes the parameter distributions, revealing the evolution of our model’s capacity to capture the underlying function space. With limited training data (16 or 32 samples), the estimated parameter distributions exhibit narrower ranges or deviate significantly from the target uniform distributions, indicating insufficient coverage of the parameter space. As the size of training dataset increases, the estimated parameter distributions progressively converge to the target uniform distributions, with the model trained on 256 samples achieving notably better alignment with the true distributions. This improvement is quantitatively validated by the decreasing mean squared error (MSE) between the generated samples and their reconstructions. These empirical results strongly support our theoretical analysis.

Kolmogorov flow

In this example, we use the proposed FunDiff to generate divergence-free velocity fields, a fundamental requirement for ensuring the physical consistency of incompressible flow simulations in computational fluid dynamics. We consider two scenarios: unconditional flow generation and flow reconstruction from partial observations. To evaluate our approach, we use the two-dimensional Kolmogorov flow, a well-established benchmark for analyzing transition to turbulence and the stability of

incompressible fluid systems. The governing equations are formulated in the vorticity-velocity representation as follows:

$$\begin{aligned}\frac{\partial \omega(x,t)}{\partial t} + \mathbf{u}(x,t) \cdot \nabla \omega(x,t) &= \frac{1}{\text{Re}} \nabla^2 \omega(x,t) + f(x), \\ \nabla \cdot \mathbf{u}(x,t) &= 0, \\ \omega(x,0) &= \omega_0(x),\end{aligned}$$

where ω is the vorticity, $\mathbf{u} = (u, v)$ is the velocity field, and Re is the Reynolds number, set to 1000. The forcing term $f(x)$ drives the flow and $x = [x_1, x_2]$. The equations are defined with periodic boundary conditions in the spatial domain $x \in (0, 2\pi)^2$ and time interval $t \in (0, T]$. The forcing term is defined as

$$f(x) = -4\cos(4x_2) - 0.1\omega(x,t),$$

where the additional drag force term $0.1\omega(x,t)$ prevents energy accumulation on large scales.

To strictly impose periodic boundary conditions and ensure the divergence-free condition, we modify the forward pass of the FAE decoder as follows:

$$[x, y, \mathcal{E}_\theta(f)] \xrightarrow{\mathcal{D}_\theta} \psi(\gamma(x, y)), \quad (3)$$

where $\mathcal{D}_\theta$ denotes the forward pass of the decoder, and the coordinate embedding γ is defined as

$$\gamma(x, y) = [\cos(x), \sin(x), \cos(y), \sin(y)]. \quad (4)$$

This embedding guarantees that the output functions have a periodicity of 2π within $[0, 2\pi]^2$. An incompressible velocity field is then computed as $(u, v) = (\partial\psi/\partial y, -\partial\psi/\partial x)$.

Unconditional flow generation

Following the training procedure in the Method section, we present our results in Fig. 3a, which displays representative generated samples along with their corresponding divergence fields, computed via automatic differentiation. The predicted divergence values are negligible, confirming the effectiveness of our approach. We also compare the spectral energy distributions between the training dataset and 1,000 generated samples (Fig. 3b). The spectra show excellent agreement up to wavenumbers of $\mathcal{O}(10^2)$. However, the model leads to some artificial high-frequency components at larger wavenumbers.

To demonstrate the importance of the divergence-free constraint, we conduct a comparative study using an unconstrained FAE trained directly on the velocity field. In this case, the forward pass is formulated as

$$[x, y, \mathcal{E}_\theta(f)] \xrightarrow{\mathcal{D}_\theta} [u(x, y), v(x, y), \omega(x, y)].$$

This unconstrained approach results in significantly higher divergence values (Fig. S1). Additionally, the spectral energy shows some discrepancies compared to the model constrained by the exact divergence-free condition.

Flow reconstruction from partial observations

For our second case study, we evaluate the model's capability to reconstruct flow fields from corrupted partial observations. We use 90% of the data for training and the rest for testing. To improve resolution-invariant reconstruction capabilities, we implement a dynamic downsampling strategy during the training of the FunDiff components. Specifically, at each training iteration, the input velocity field is randomly downsampled by factors of 1, 2, or 4. We first examine the impact of the divergence-free constraint by comparing FunDiff models with and without this condition (Fig. 3e). Both variants demonstrate robust performance across different downsampling factors and noise levels. Notably, the divergence-constrained FunDiff model consistently achieves lower test errors, validating the importance of physical constraints in the reconstruction process.

Furthermore, we benchmark our approach against several state-of-the-art neural operators and PDE surrogate models, including FNO¹³, ViT⁸, and UNet⁶⁰, as well as more recent architectures such as CViT⁶¹, DPOT⁶², CNextU-Net⁶³, and AViT⁶⁴. To further evaluate performance under inverse problem settings, we also include three diffusion-based baselines: Diffusion Posterior Sampling (DPS)⁶⁵, DiffusionPDE⁴², and Spatially-Aware Diffusion⁵¹. Detailed implementation specifications for these baseline models are provided in Appendix S4.1. Fig. 3c presents reconstructed flow fields from various models, where the input consists of 64×64 resolution velocity fields corrupted by 20% Gaussian noise. It can be observed that neural operator baselines tend to produce highly discontinuous or spatially inconsistent patterns, whereas diffusion-based models recover coherent large-scale structures but often lose fine-scale features. The results demonstrate that our model uniquely achieves both smooth and accurate reconstructions. Additional visualizations of our model's predictions, with and without the divergence-free condition imposed in FAE decoder, are shown in Figs. S2 and S3, respectively. The quantitative comparison using relative L^2 errors across different downsampling factors is presented in Fig. 3e. These results confirm that our FunDiff model maintains consistent performance across all tested resolutions while exhibiting strong resilience to noise.

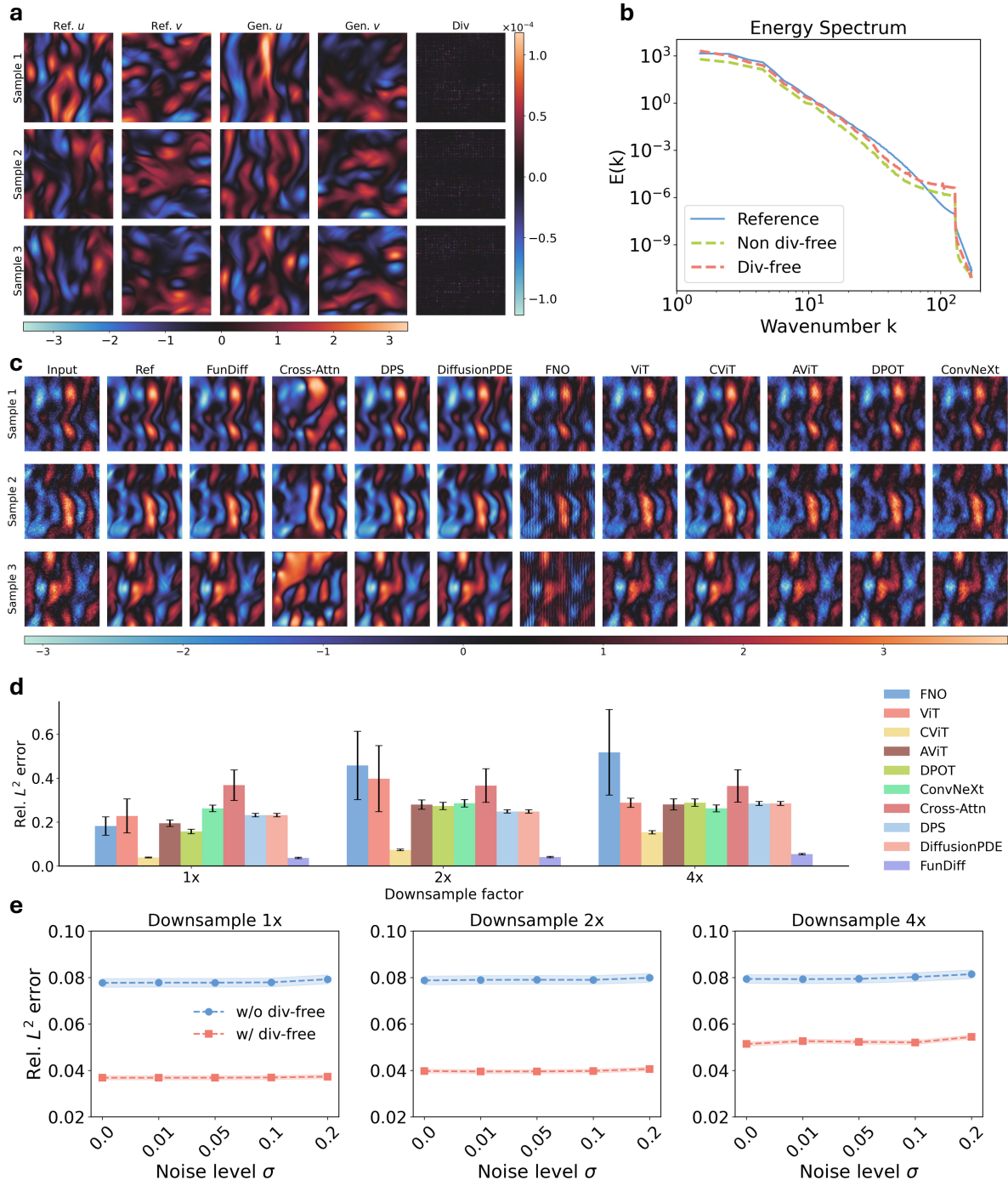


Figure 3. Kolmogorov flow. (a) Unconditional generation with enforced divergence-free constraint. (b) Comparison of spectral energy distributions between models constrained with and without divergence-free conditions. (c) Flow field reconstruction from noisy, low-resolution observations. Visualization of reconstructed velocity fields using different methods with input data of 64×64 resolution corrupted by 20% Gaussian noise. (d) Comparative analysis of reconstruction errors across methods at various downsampling factors. (e) Relative L^2 error of reconstructed velocity fields across varying noise levels (σ) for different spatial resolution reductions (1x, 2x, and 4x downsampling). Dashed lines show mean performance with (blue) and without (red) PDE constraints, while shaded regions indicate one standard deviation.

Burgers' equation

As our next example, we demonstrate the effectiveness and flexibility of our framework in generating physically consistent samples through the incorporation of physical laws as soft constraints in the training of FAE. To this end, we consider the Burgers' equation, which serves as a fundamental prototype for conservation laws that can develop discontinuities. The governing equation is given by

$$\frac{du}{dt} + u \frac{du}{dx} - \nu \frac{d^2u}{dx^2} = 0, \quad (x, t) \in (0, 1) \times (0, 1], \quad (5)$$

$$u(x, 0) = a(x), \quad x \in (0, 1) \quad (6)$$

with periodic boundary conditions, where $\nu = 0.001$ denotes the viscosity coefficient, and the initial condition $a(x)$ is sampled from a Gaussian random field (GRF) satisfying periodic boundary conditions.

Let u_θ denote the FAE output. We define the PDE residual as

$$\mathcal{R}[u_\theta](x, t) = \frac{du_\theta(x, t)}{dt} + u_\theta(x, t) \frac{du_\theta(x, t)}{dx} - \nu \frac{d^2u_\theta(x, t)}{dx^2}.$$

The FAE is trained by minimizing a composite loss

$$\mathcal{L}(\theta) := \underbrace{w_{data} \frac{1}{NM} \sum_{i=1}^N \sum_{j=1}^M |u_\theta^i(x_j, t_j) - u^i(x_j, t_j)|^2}_{\mathcal{L}_{data}(\theta)} + \underbrace{w_{physics} \frac{1}{NM} \sum_{i=1}^N \sum_{j=1}^M |\mathcal{R}[u_\theta^i](x_j, t_j)|^2}_{\mathcal{L}_{physics}(\theta)}.$$

Here, for each sample u^i , we randomly sample query coordinates $\{(x_j, t_j)\}_{j=1}^M$ from the uniform grid during each gradient descent iteration. All network derivatives are computed using automatic differentiation. Specifically, we set $w_{data} = 100, w_{physics} = 1$ and randomly downsample u^i by a factor of 1, 2, 5 at each iteration of gradient descent.

Similar to the Kolmogorov flow example, our objective here is to train a FunDiff model to reconstruct the solution field from sparse observations. We evaluate our model against established baseline methods. We show three examples in Fig. 4a. Neural operator baselines generally produce blurry and discontinuous reconstructions, whereas diffusion-based approaches yield smoother results but often exhibit small oscillations near sharp gradients. In contrast, our FunDiff generates sharp and smooth reconstructions, which are in excellent agreement with the reference numerical solutions. Fig. 4b presents a quantitative comparison of model performance across inputs with varying downsampling factors and 20% Gaussian noise corruption. Notably, FunDiff outperforms all baselines. Additional examples of our model's predictions are shown in Fig. S4. Their physical consistency is confirmed by low PDE residuals, remaining on the order of $\mathcal{O}(10^{-2})$ across the entire domain. We also systematically examine the effect of noise and data sparsity (Fig. 4c). As expected, reconstruction error increases with higher noise levels, but FunDiff demonstrates strong robustness to data sparsity, maintaining stable performance even under aggressive downsampling (factors up to 5).

To rigorously evaluate the impact of physical constraints, we conduct an ablation study by training the function encoder without the physics loss term $\mathcal{L}_{physics}$. The absence of PDE-based regularization leads to significantly higher predictive errors and PDE residuals in the generated solutions (Fig. 4d). This degradation in physical consistency is further visualized through the generated samples in Fig. S5, underscoring the crucial role of physics-informed training in our framework.

Linear elasticity

Symmetries play an important role in physics, providing a framework for understanding conservation laws and invariances that govern natural phenomena. This example aims to demonstrate how to embed prior symmetries into FunDiff models. To this end, we consider linear elasticity in computational micromechanics, which involves solving the Lippmann-Schwinger equation⁶⁶ in a 2D periodic representative volume element (RVE) microstructural domain $\Omega \subset \mathbb{R}^2$ with spatial coordinates $\mathbf{x} = (x, y)$ under plane strain conditions. The problem is formulated as

$$\begin{cases} \epsilon_{ij}(\mathbf{x}) + \int_{\Omega} G_{ijkl}^{(0)}(\mathbf{x}, \mathbf{x}') : [\sigma_{ij}(\mathbf{x}') - C_{ijkl}^0 : \epsilon_{kl}(\mathbf{x}')] d\mathbf{x}' - \bar{\epsilon}_{ij} = 0, & \mathbf{x} \in \Omega, \\ \sigma_{ij}(\mathbf{x}) = C_{ijkl}(\mathbf{x}) : \epsilon_{kl}(\mathbf{x}). \end{cases} \quad (7)$$

These relations represent balance and constitutive equations, respectively, which are written in a component-wise manner ($1 \leq i, j, k, l \leq 2$) using the Voigt notation. $\mathbf{x}$ and $\mathbf{x}'$ denote a pair of target and source points inside the RVE domain Ω . $\sigma_{ij}, \epsilon_{ij}$

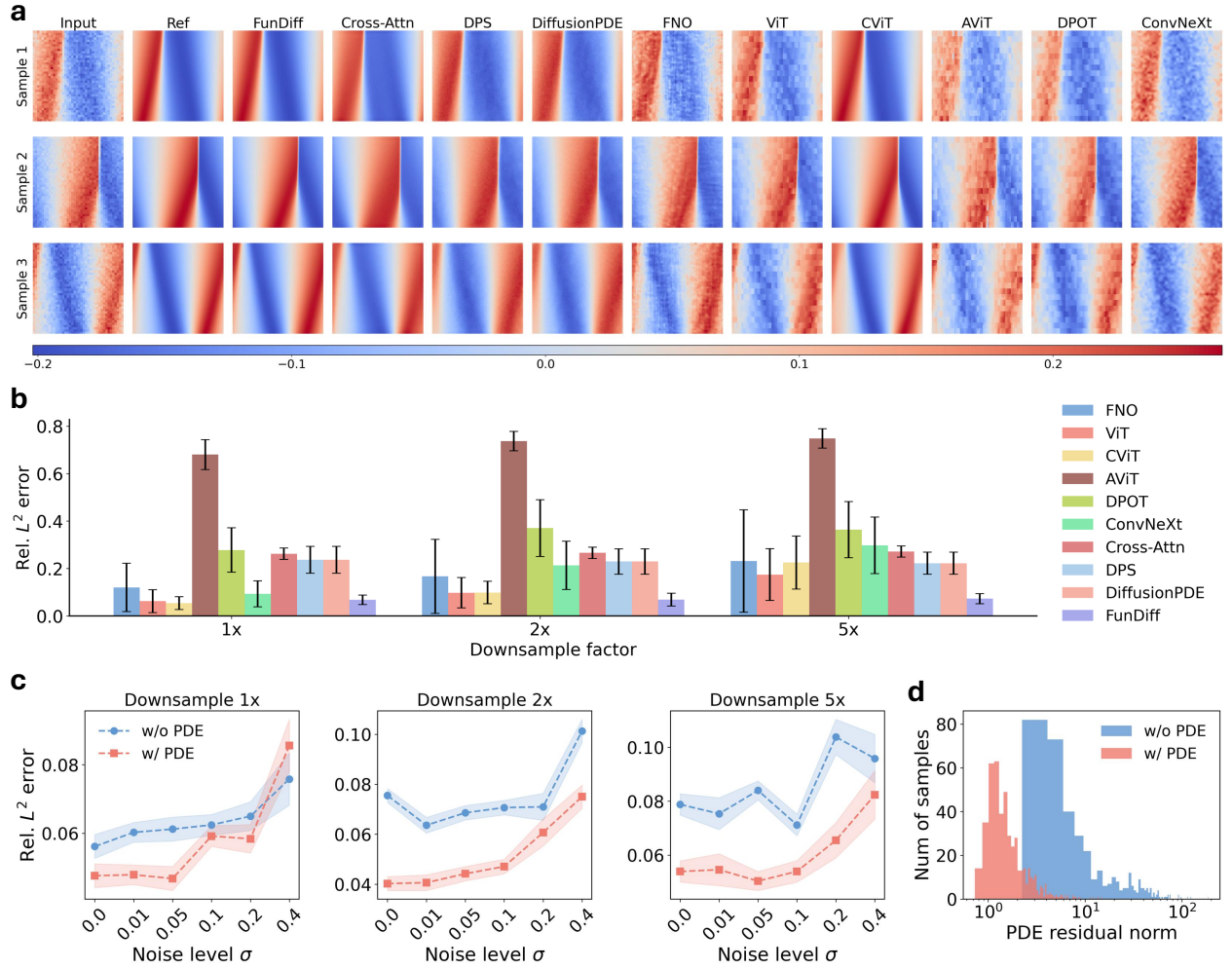


Figure 4. Burgers' equation. (a) Solution reconstruction from noisy, low-resolution observations. Visualization of reconstructed solutions using different methods with input data of 40×40 resolution corrupted by 20% Gaussian noise. (b) Comparative analysis of reconstruction errors across methods at various downsampling factors. (c) Relative L^2 error of reconstructed solutions across varying noise levels (σ) for different spatial resolution reductions (1x, 2x, and 5x downsampling). Dashed lines show mean performance with (blue) and without (red) PDE constraints, while shaded regions indicate one standard deviation. (d) Histogram of PDE residual norms, comparing the physical consistency of solutions with and without PDE constraints, demonstrating improved physical consistency when PDE constraints are incorporated.

and C_{ijkl} denote stress, strain, and fourth-order elastic tensor, respectively. $C_{ijkl}(\mathbf{x})$ is expressed in terms of local Young's modulus $E(\mathbf{x})$ and Poisson ratio $\nu(\mathbf{x})$ as

$$C_{ijkl}(\mathbf{x}) = \frac{E(\mathbf{x})\nu(\mathbf{x})}{(1+\nu(\mathbf{x}))(1-2\nu(\mathbf{x}))} \delta_{ij}\delta_{kl} + \frac{E(\mathbf{x})}{2(1+\nu(\mathbf{x}))} (\delta_{ik}\delta_{jl} + \delta_{il}\delta_{jk}),$$

where δ_{ij} denotes the Kronecker delta function. $G_{ijkl}^{(0)}$ denotes Green's operator corresponding to the homogeneous reference material with elastic tensor C_{ijkl}^0 ⁶⁷. $\bar{\epsilon}_{ij}$ denotes far field strain used to apply the strain-controlled boundary condition, equal to the average strain over the RVE domain Ω . In this example, we model the point-wise strain concentration tensor $A_{ijkl}(\mathbf{x}) \in \mathbb{R}^{3 \times 3}$, which linearly maps $\bar{\epsilon}_{ij}$ to $\epsilon_{ij}(\mathbf{x})$ independent of loading conditions:

$$\epsilon_{ij}(\mathbf{x}) = A_{ijkl}(\mathbf{x}) : \bar{\epsilon}_{kl}.$$

$A_{ijkl}(\mathbf{x})$ can be computed by triply solving (7) subject to three orthogonal unit far field strain $\bar{\epsilon}_{ij}$ as

$$\bar{\epsilon}_{ij} = \{[1, 0, 0]^T, [0, 1, 0]^T, [0, 0, 1]^T\}. \quad (8)$$

Due to the intrinsic symmetry of the RVE configuration, the resulting strain concentration tensor field $A_{ijkl}(\mathbf{x})$ exhibits a symmetric pattern with respect to both $x = \frac{1}{2}$ and $y = \frac{1}{2}$. We aim to train a FunDiff model to reconstruct the strain concentration tensor field from sparse observations while respecting the inherent symmetry priors. To this end, we modify the forward pass of the decoder to enforce these constraints. Let $\mathbf{z}$ denote the FAE encoder output latents. The symmetry-preserved decoder $\hat{\mathcal{D}}_\theta$ is then constructed as

$$\hat{\mathcal{D}}_\theta(\mathbf{z}, x, y) = \mathcal{D}_\theta(\mathbf{z}, x, y) + \mathcal{D}_\theta(\mathbf{z}, 1-x, y) + \mathcal{D}_\theta(\mathbf{z}, x, 1-y) + \mathcal{D}_\theta(\mathbf{z}, 1-x, 1-y).$$

It can be easily shown that this formulation ensures that $\hat{\mathcal{D}}_\theta$ generates functions with perfect symmetry about the lines $x = 1/2$ and $y = 1/2$. Although here we only focus on such a relative simple symmetry for illustration purposes, it is worth noting here that more general symmetries can be embedded in a similar manner⁶⁸.

Fig. 5a presents reconstructed samples generated by our FunDiff model from a coarse strain concentration field corrupted by 10% Gaussian noise. The results show that our model preserves underlying symmetries and accurately resolves sharp material interfaces. To assess robustness against different types of noise, we report test errors in Fig. 5b. The consistently low errors across all noise configurations highlight the framework's reliability in practical applications.

To further demonstrate the critical role of the continuous decoder in our framework, we implement a baseline using a discrete autoencoder with a Vision Transformer encoder and an MLP decoder projecting to the target pixel space. We train a diffusion transformer under identical conditions, with representative generated samples shown in Fig. S6. Unlike our approach, the baseline fails to preserve symmetry or enforce physical continuity in strain fields, instead producing piecewise discontinuous predictions.

Turbulence mass transfer

In the last example, we examine the model capacity of inferring non-intrusive quantities such as pressure in turbulent problems. Specifically, we consider turbulent mass transfer in a rectangular channel. This phenomenon plays a critical role in the chemical engineering process, directly influencing reaction kinetics, conversion efficiency, and product purity. Our study focuses on a computational domain $\Omega = \{(x, y) \mid 0.006 < x < 0.018, 0 < y < 0.01\}$. Within this domain, we are interested in training a FunDiff model to infer the obstacle topology and pressure field given the sparse and noisy velocity measurements.

The simulation employs a two-equation model for turbulent mass transfer based on the Reynolds-averaged Navier–Stokes (RANS) equations⁶⁹, which can be expressed as

$$\begin{aligned} \frac{\partial(\rho u_i)}{\partial x_i} &= 0, \\ u_i \frac{\partial(\rho u_j)}{\partial x_i} &= -\frac{\partial P}{\partial x_j} + \frac{\partial}{\partial x_i} \left[(\mu_m + \mu_t) \frac{\partial u_j}{\partial x_i} \right], \\ u_i \frac{\partial(\rho C)}{\partial x_i} &= \frac{\partial}{\partial x_i} \left[\rho (D_m + D_t) \frac{\partial C}{\partial x_i} \right], \end{aligned}$$

where p, u_i, x_i are the pressure, velocity and location respectively; the subscript i represents the coordinate direction which can take x and y for the two-dimensional system. Here, C is the time-averaged concentration; ρ, μ_m and D_m are density,

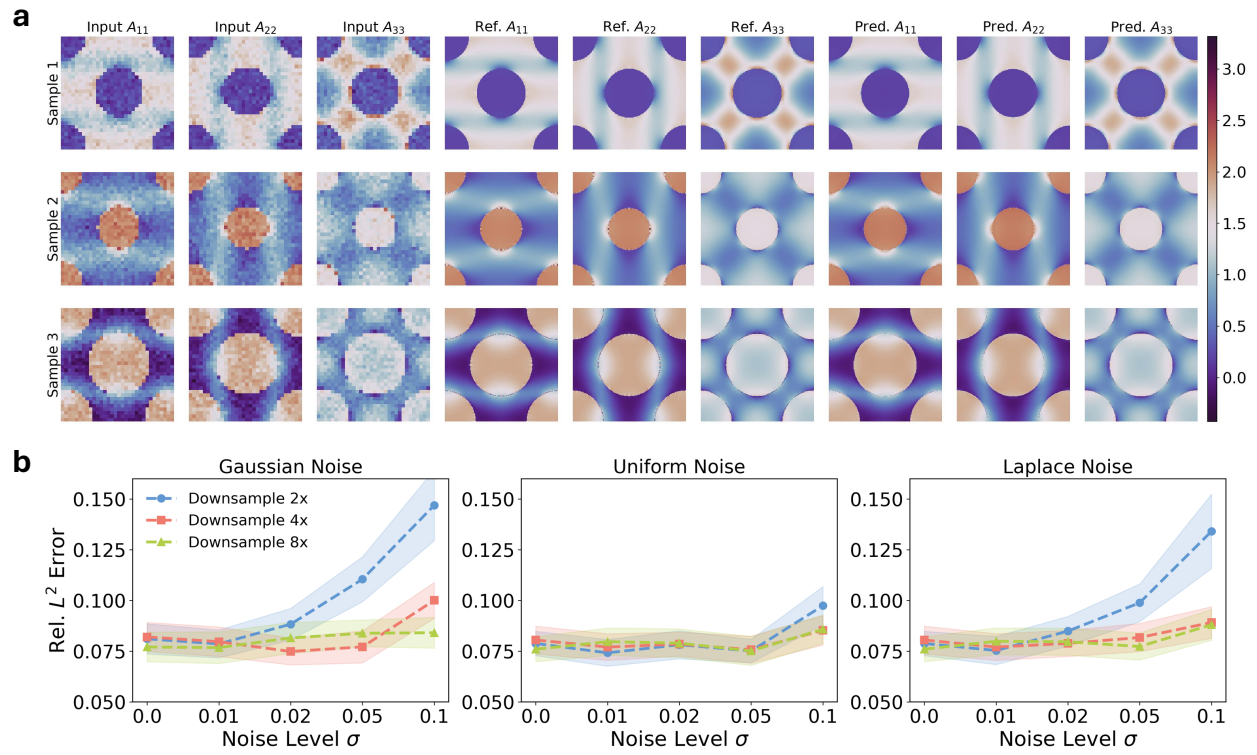


Figure 5. Linear elasticity. (a) Comparison of diagonal strain concentration tensor distributions (Voigt notation) between provided coarse measurements (32×32) and reconstructions from the trained FunDiff model. The generated samples maintain enforced symmetry constraints and achieve a relative L^2 error of 5.12% when compared to ground truth data at 256×256 resolution. (b) Comparison of reconstruction errors under different noise types and downsampling rates.

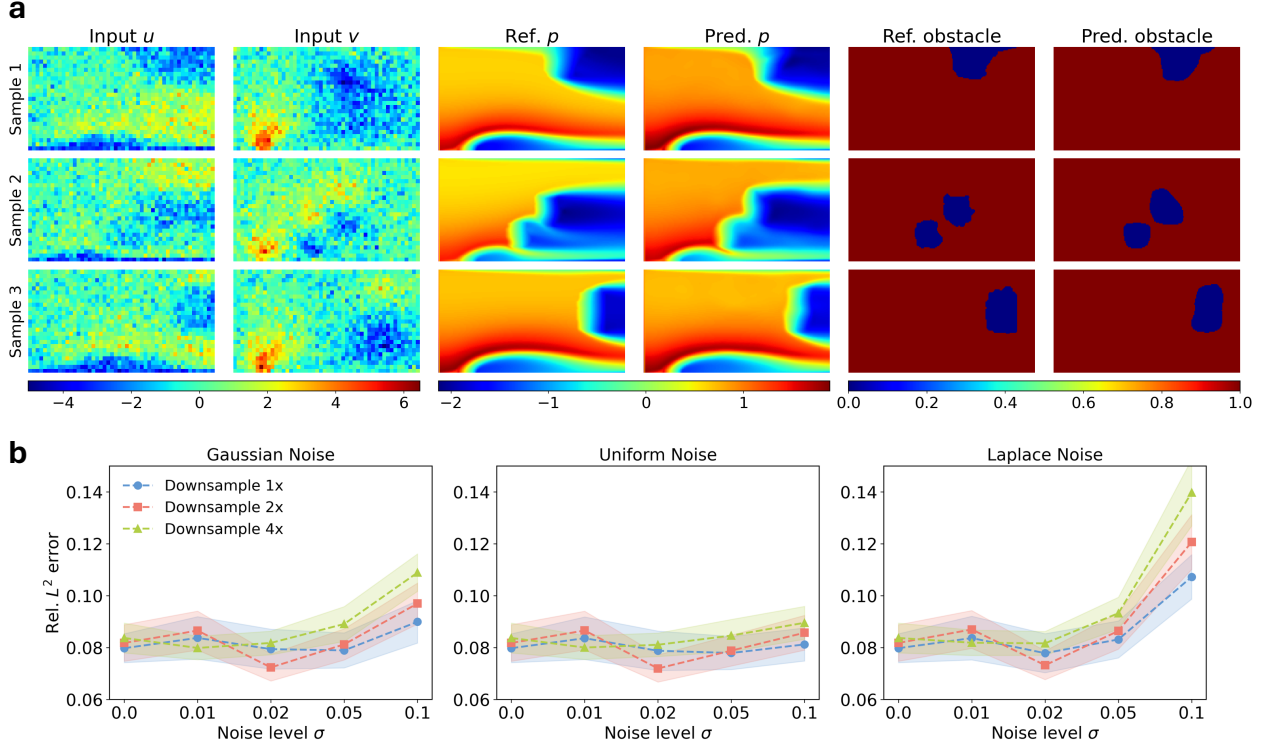


Figure 6. Turbulent mass transfer. (a) Left: Input velocity fields at low resolution with 10% noise corruption. Middle: Comparison between reference and generated pressure fields. Right: Comparison between reference and generated obstacle geometry. (b) Relative L^2 error comparison between generated samples and reference solutions across varying input resolutions and noise levels.

laminar viscosity, and laminar mass diffusivity, which are physical property constants and fixed to 1000 kg/m^3 , $1.003 \times 10^{-6} \text{ m}^2/\text{s}$, $1.003 \times 10^{-8} \text{ m}^2/\text{s}$, respectively. μ_t and D_t are the turbulent viscosity and turbulent mass diffusivity, respectively.

We begin by training an FAE to reconstruct the key physical variables: velocity field, pressure field, and the signed distance function (SDF) representing obstacle geometry. To enhance the capacity of the model to handle inputs of varying resolutions, we augment training by randomly downsampling inputs by factors of 1, 2, and 4 while preserving the target resolution. The latent diffusion model is then trained with latent representations of the pressure and SDF, conditioned on the encoded representations of the velocity field, obtained from the trained autoencoder’s latent space.

Representative generated samples (Fig. 6a) validate the model’s ability to infer pressure fields and reconstruct obstacle geometries despite heavily corrupted input velocity measurements. The reconstructions align closely with numerical estimations. Fig. 6b presents the test error across various downsampling factors, noise levels, and noise types. The results indicate that our model remains robust even under severe noise corruption and low-resolution inputs. Our results demonstrate the model’s effectiveness in preserving critical flow features and geometric details, and highlight the framework’s potential for real-world applications where high-fidelity reconstructions are required from sparse or noisy measurements.

Discussion

In this work, we propose FunDiff, a flexible and efficient framework for generative modeling in function spaces, addressing fundamental challenges in generating continuous function data while preserving physical constraints. Our primary contributions are twofold: (1) a rigorous theoretical foundation establishing conditions for function distribution approximation via autoencoders, and (2) a novel constrained function autoencoder (FAE) that inherently enforces physical principles. Through extensive experimentation across diverse physical systems, we have demonstrated that our approach successfully generates physically valid samples while maintaining high fidelity to the target PDE solution distributions.

While our current work lays a strong foundation for physics-informed generative modeling, several promising directions for future research emerge. One particularly interesting direction is to develop not purely data-driven, but physics-informed diffusion-based PDE solvers. This could be achieved by incorporating PDE residuals directly into the score function of diffusion

models and by using additional initial and boundary conditions as guidance during sampling, thereby steering the generated samples toward the corresponding PDE solutions. Another natural extension would be to adapt our framework to handle more complex geometries, thereby enabling broader applicability in real-world scientific and engineering problems. Handling complex domains necessitates incorporating geometric inductive biases, such as mesh-based representations, implicit neural surfaces, or graph-based function approximations. Moreover, extending our approach to multiphysics applications represents a critical and exciting frontier. Many engineering and scientific challenges involve the interplay of multiple physical processes, such as fluid-structure interactions, thermo-mechanical coupling, and electromagnetic-thermal phenomena, which require models that can simultaneously capture and reconcile diverse physical constraints. These advancements will significantly expand the scope of generative modeling for physical systems and ultimately contribute to more accurate and efficient simulation-based design across disciplines.

Methods

To realize our theoretical framework for generative modeling in function spaces, we develop FunDiff, a practical implementation that addresses the key challenges of handling infinite-dimensional function distributions while preserving physical constraints. Our approach must accommodate discretized functions at varying resolutions, enable continuous evaluation at arbitrary coordinates, and flexibly incorporate domain-specific physical priors without compromising the generative modeling process. The architecture is designed to decouple physical constraint enforcement from diffusion training, ensuring stable optimization while maintaining mathematical rigor in constraint satisfaction. We detail the FunDiff network architecture, training procedures, and constraint integration mechanisms that enable robust function generation across diverse scientific domains.

Network architecture

The FAE processes discretized functions $\mathbf{f} \in \mathbb{R}^{n \times d}$, where n denotes the spatial discretization resolution and d the codomain dimension. The FunDiff architecture comprises three core components: (1) a resolution-invariant encoder $\mathcal{E}$ combining a Vision Transformer (ViT)⁸ and a Perceiver module⁹, (2) a latent diffusion model operating on the compact latent space, and (3) a physics-aware decoder $\mathcal{D}$ that reconstructs continuous functions.

Encoder. The encoder $\mathcal{E}$ employs a ViT backbone to extract features and a Perceiver module to handle functions of different discretizations. Input functions $\mathbf{f} \in \mathbb{R}^{H \times W \times C}$ are first processed by ViT-style patchification, resulting in flattened patches $\mathbf{f}_p \in \mathbb{R}^{L \times D}$, where $L = \frac{H}{P} \times \frac{W}{P}$ represents the length of sequence, P is the size of the patch, and D denotes the embedding dimension. To accommodate varying resolutions, trainable positional embeddings (PE) are dynamically interpolated to match the input sequence length L , yielding position-aware patch embeddings $\mathbf{f}_{pe} = \mathbf{f}_p + \text{Interp}(\text{PE}, L)$.

A perceiver-style cross-attention module maps these variable-length inputs to fixed-dimensional latent representations. Specifically, we learn a predefined number of latent input queries $\mathbf{z}_q \in \mathbb{R}^{N \times D}$. These queries serve as learnable parameters in a cross-attention module, which processes the visual features of our input as follows

$$\begin{aligned} \mathbf{z}' &= \mathbf{z}_q + \text{MHA}(\text{LN}(\mathbf{z}_q), \text{LN}(\mathbf{f}_{pe}), \text{LN}(\mathbf{f}_{pe})), \\ \mathbf{z}_{agg} &= \mathbf{z}' + \text{MLP}(\text{LN}(\mathbf{z}')), \end{aligned}$$

where MHA, LN, and MLP represent multi-head self-attention, layer normalization, and a multi-layer perceptron, respectively. We further process the aggregated tokens $\mathbf{z}_{agg}$ using a sequence of L pre-norm Transformer blocks^{70,71},

$$\begin{aligned} \mathbf{z}_0 &= \text{LN}(\mathbf{z}_{agg}), \\ \mathbf{z}'_\ell &= \text{MSA}(\text{LN}(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, & \ell = 1, \dots, L, \\ \mathbf{z}_\ell &= \text{MLP}(\text{LN}(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, & \ell = 1, \dots, L, \end{aligned}$$

where MSA is a standard multi-head self-attention module used to model long-range dependencies across the sequence.

Decoder. The decoder module draws inspiration from the Continuous Vision Transformer (CViT)⁷², which enables continuous evaluation at any query coordinate through a cross-attention mechanism between query coordinates and encoded features. We start by embedding coordinates into the latent space, which can be efficiently achieved through either random Fourier features⁷³ or grid-based embeddings⁷².

The core of our decoder uses embedded coordinates $\mathbf{x}_0 \in \mathbb{R}^D$ as queries, with the encoder output $\mathbf{z}_L$ serving as both keys and values in a series of K cross-attention Transformer blocks:

$$\begin{aligned} \mathbf{x}'_k &= \mathbf{x}_{k-1} + \text{MHA}(\text{LN}(\mathbf{x}_{k-1}), \text{LN}(\mathbf{z}_L), \text{LN}(\mathbf{z}_L)), & k = 1 \dots K, \\ \mathbf{x}_k &= \mathbf{x}'_k + \text{MLP}(\text{LN}(\mathbf{x}'_k)), & k = 1 \dots K. \end{aligned}$$

Finally, a small feedforward network projects the features $\mathbf{x}_K$ to the desired output dimension. This architecture naturally extends to other function representations (e.g., point clouds via PointNet⁷⁴ or meshes via graph neural networks⁷⁵) through appropriate input processing modules.

Latent diffusion with conditional Diffusion Transformer. We implement the diffusion process in the learned latent space using a Diffusion Transformer (DiT)⁷⁶. We condition the model on coarse solution measurements $\mathbf{c}$ by encoding them through the pretrained FAE encoder $\mathcal{E}_\theta$. The resulting latent-space guidance vectors are additively combined with the diffusion inputs:

$$\tilde{\mathbf{z}}_1 = \mathbf{z}_1 + \mathcal{E}_\theta(\mathbf{c}).$$

The DiT backbone consists of a sequence of Transformer blocks that utilize AdaLN-Zero modulation⁷⁶, an enhanced variant of Adaptive Layer Normalization designed for stable time-dependent conditioning. For each block, the modulation parameters are computed as

$$\begin{aligned}\gamma, \beta &= \text{MLP}(t), \\ \mathbf{z}' &= \text{LN}(\mathbf{z}) \odot \gamma + \beta,\end{aligned}$$

where t represents the timestep. Following the approach in Ref.⁷⁶, we incorporate element-wise scaling factors α , initialized to zero, in the residual pathways:

$$\mathbf{z}_{\text{out}} = \alpha \odot \text{MSA}(\mathbf{z}') + \mathbf{z}.$$

By initializing these scaling factors to zero, each Transformer block initially acts as an identity transform. This strategy enables gradual learning of temporal and conditional modulations while maintaining stable gradient flow during the early stages of training.

Training. Our training procedure consists of two stages. In the first stage, we train an FAE to learn a compact and expressive latent representation of functions. We consider a set of ground truth functions $\{f_i\}_{i=1}^N$, where each function $f_i: \mathcal{X} \rightarrow \mathbb{R}$ maps input coordinates $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^d$ to scalar outputs. For training, each function f_i is discretized at a fixed set of input locations $\{\mathbf{x}_j\}_{j=1}^M$, resulting in a vector of evaluations $\mathbf{f}_i = [f_i(\mathbf{x}_1), \dots, f_i(\mathbf{x}_M)] \in \mathbb{R}^M$. The reconstruction loss is given by

$$\mathcal{L}_{\text{FAE}} = \frac{1}{NM} \sum_{i=1}^N \sum_{j=1}^M |f_i(\mathbf{x}_j) - \mathcal{D}(\mathcal{E}(\mathbf{f}_i))(\mathbf{x}_j)|^2,$$

where $\mathcal{E}$ and $\mathcal{D}$ denote the encoder and decoder respectively, and $f_i(\mathbf{x}_j)$ represents the value of a ground truth function f_i at query point $\mathbf{x}_j$. During training, we randomly sample functions $\{f_i\}$ and query coordinates $\{\mathbf{x}_j\}_{j=1}^M$ at each iteration.

In the second stage, we freeze the pre-trained FAE and train the DiT model in the induced latent space. Following the rectified flow framework, the training objective is formulated as

$$\mathcal{L}_{\text{DiT}} = \int_0^1 \mathbb{E}_{\mathbf{f} \sim p_{\text{data}}} [|\mathbf{z}_1 - \mathbf{z}_0 - \mathbf{g}(\mathbf{z}_\tau, \tau)|^2] d\tau, \quad \text{with } \mathbf{z}_\tau = (1 - \tau)\mathbf{z}_1 + \tau\mathbf{z}_0, \quad \mathbf{z}_1 = \mathcal{E}(\mathbf{f}), \quad (9)$$

where $\mathbf{z}_1$ is obtained by encoding the input function through the pre-trained FAE encoder, $\mathbf{z}_0 \sim \mathcal{N}(0, \mathbf{I})$ is sampled from the standard normal distribution, and $\mathbf{g}(\mathbf{z}_\tau, \tau)$ is parameterized by our DiT model predicting the velocity field. In practice, we approximate the time integral by uniformly sampling $\tau \sim \mathcal{U}[0, 1]$ at each training step.

Inference. At inference, we follow a straightforward two-step generation process. First, we sample latent codes from our learned diffusion model by solving the ODE using a standard ODE solver (e.g., Runge-Kutta method):

$$\frac{d\mathbf{z}(t)}{dt} = \mathbf{g}(\mathbf{z}(t), t), \quad t \in [0, 1],$$

starting from a sample of standard Gaussian noise $\mathbf{z}_1 \sim \mathcal{N}(0, \mathbf{I})$ and integrating backward to obtain $\mathbf{z}_0$. This process is deterministic and typically requires 20–50 function evaluations for high-quality samples. Second, we decode the sampled latent codes through the FAE decoder to obtain the final continuous function. Importantly, since our decoder operates on arbitrary query points, we can evaluate the generated function at any desired resolution or location in the domain.

Implementation details. We maintain consistent architectural choices and training procedures across all experiments unless explicitly stated otherwise. Our FAE architecture comprises an 8-layer encoder and a 4-layer decoder, both utilizing an embedding dimension of 256 and MLP width of 512. The encoder’s patch size is adaptively selected based on input resolution. The DiT employs 8 Transformer blocks with an embedding dimension of 256 and MLP width of 512. All attention operations use 8 heads.

Training protocol. For the FAE training, we employ the AdamW optimizer⁷⁷ with a weight decay of 10^{-5} . The learning rate follows a two-phase schedule: an initial linear warm-up from zero to 10^{-3} over 2,000 steps, followed by exponential decay with a factor of 0.9 every 2,000 steps. Each training batch contains 16 samples, with 4,096 query coordinates and their corresponding outputs randomly sampled from the grid. To enhance the model’s resolution invariance, we randomly downsample inputs by different factors (e.g., 1, 2, 4, 8) during training while optimizing the model to reconstruct the original high-resolution input function by minimizing the MSE loss.

The DiT follows a similar optimization strategy but operates with larger batches of 128 samples over 10^5 iterations. For training the conditioned diffusion models, we generate conditions by randomly downsampling input functions and encoding them through the pretrained FAE encoder. Detailed hyperparameters for all experiments are provided in Appendix S4.

References

1. Kingma, D. P. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).
2. Goodfellow, I. *et al.* Generative adversarial networks. *Commun. ACM* **63**, 139–144 (2020).
3. Ho, J., Jain, A. & Abbeel, P. Denoising diffusion probabilistic models. *Adv. neural information processing systems* **33**, 6840–6851 (2020).
4. Song, Y. *et al.* Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456* (2020).
5. Lipman, Y., Chen, R. T., Ben-Hamu, H., Nickel, M. & Le, M. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747* (2022).
6. Liu, X., Gong, C. & Liu, Q. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003* (2022).
7. Saharia, C. *et al.* Photorealistic text-to-image diffusion models with deep language understanding. *Adv. neural information processing systems* **35**, 36479–36494 (2022).
8. Dosovitskiy, A. *et al.* An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
9. Jaegle, A. *et al.* Perceiver: General perception with iterative attention. In *International conference on machine learning*, 4651–4664 (PMLR, 2021).
10. Kong, Z., Ping, W., Huang, J., Zhao, K. & Catanzaro, B. Diffwave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761* (2020).
11. Wu, K. E. *et al.* Protein structure generation via folding diffusion. *Nat. communications* **15**, 1059 (2024).
12. Xu, M. *et al.* Geodiff: A geometric diffusion model for molecular conformation generation. *arXiv preprint arXiv:2203.02923* (2022).
13. Li, Z. *et al.* Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations* (2021).
14. Lu, L., Jin, P., Pang, G., Zhang, Z. & Karniadakis, G. E. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **3**, 218–229 (2021).
15. Jin, P., Meng, S. & Lu, L. Mionet: Learning multiple-input operators via tensor product. *SIAM J. on Sci. Comput.* **44**, A3490–A3514 (2022).
16. Lu, L. *et al.* A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. *Comput. Methods Appl. Mech. Eng.* **393**, 114778 (2022).
17. Jiao, A., He, H., Ranade, R., Pathak, J. & Lu, L. One-shot learning for solution operators of partial differential equations. *arXiv preprint arXiv:2104.05512* (2021).
18. Wu, Z. *et al.* Coast: Intelligent time-adaptive neural operators. *arXiv preprint arXiv:2502.08574* (2025).
19. Zhang, H., Liu, L. & Lu, L. Federated scientific machine learning for approximating functions and solving differential equations with data heterogeneity. *arXiv preprint arXiv:2410.13141* (2024).
20. Park, J. J., Florence, P., Straub, J., Newcombe, R. & Lovegrove, S. DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 165–174 (2019).

21. Mildenhall, B. *et al.* Nerf: Representing scenes as neural radiance fields for view synthesis. *Commun. ACM* **65**, 99–106 (2021).
22. Dupont, E., Kim, H., Eslami, S., Rezende, D. & Rosenbaum, D. From data to functa: Your data point is a function and you can treat it like one. *arXiv preprint arXiv:2201.12204* (2022).
23. Sitzmann, V., Martel, J., Bergman, A., Lindell, D. & Wetzstein, G. Implicit neural representations with periodic activation functions. *Adv. Neural Inf. Process. Syst.* **33**, 7462–7473 (2020).
24. Zhang, B., Tang, J., Niessner, M. & Wonka, P. 3dshape2vecset: A 3d shape representation for neural fields and generative diffusion models. *ACM Transactions On Graph. (TOG)* **42**, 1–16 (2023).
25. Jun, H. & Nichol, A. Shap-e: Generating conditional 3d implicit functions. *arXiv preprint arXiv:2305.02463* (2023).
26. Erkoç, Z., Ma, F., Shan, Q., Nießner, M. & Dai, A. Hyperdiffusion: Generating implicit neural fields with weight-space diffusion. In *Proceedings of the IEEE/CVF international conference on computer vision*, 14300–14310 (2023).
27. Dutordoir, V., Saul, A., Ghahramani, Z. & Simpson, F. Neural diffusion processes. In *International Conference on Machine Learning*, 8990–9012 (PMLR, 2023).
28. Phillips, A. *et al.* Spectral diffusion processes. *arXiv preprint arXiv:2209.14125* (2022).
29. Hui, K.-H., Li, R., Hu, J. & Fu, C.-W. Neural wavelet-domain diffusion for 3d shape generation. In *SIGGRAPH Asia 2022 Conference Papers*, 1–9 (2022).
30. Bautista, M. A. *et al.* Gaudi: A neural architect for immersive 3d scene generation. *Adv. Neural Inf. Process. Syst.* **35**, 25102–25116 (2022).
31. Chou, G., Bahat, Y. & Heide, F. Diffusion-sdf: Conditional generative modeling of signed distance functions. In *Proceedings of the IEEE/CVF international conference on computer vision*, 2262–2272 (2023).
32. Dupont, E., Teh, Y. W. & Doucet, A. Generative models as distributions of functions. *arXiv preprint arXiv:2102.04776* (2021).
33. Kerrigan, G., Ley, J. & Smyth, P. Diffusion generative models in infinite dimensions. *arXiv preprint arXiv:2212.00886* (2022).
34. Pidstrigach, J., Marzouk, Y., Reich, S. & Wang, S. Infinite-dimensional diffusion models. *arXiv preprint arXiv:2302.10130* (2023).
35. Franzese, G. *et al.* Continuous-time functional diffusion processes. *Adv. Neural Inf. Process. Syst.* **36** (2024).
36. Lim, J. H. *et al.* Score-based diffusion models in function space. *arXiv preprint arXiv:2302.07400* (2023).
37. Kovachki, N. *et al.* Neural operator: Learning maps between function spaces with applications to pdes. *J. Mach. Learn. Res.* **24**, 1–97 (2023).
38. Jiang, Z., Zhu, M. & Lu, L. Fourier-mionet: Fourier-enhanced multiple-input neural operators for multiphase modeling of geological carbon sequestration. *Reliab. Eng. & Syst. Saf.* **251**, 110392 (2024).
39. Yin, M. *et al.* A scalable framework for learning the geometry-dependent solution operators of partial differential equations. *Nat. Comput. Sci.* **4**, 928–940 (2024).
40. Lee, J. E. *et al.* Efficient and generalizable nested fourier-deeponet for three-dimensional geological carbon sequestration. *Eng. Appl. Comput. Fluid Mech.* **18**, 2435457 (2024).
41. Li, Z. *et al.* Learning spatiotemporal dynamics with a pretrained generative model. *Nat. Mach. Intell.* **6**, 1566–1579 (2024).
42. Huang, J., Yang, G., Wang, Z. & Park, J. J. Diffusionpde: Generative pde-solving under partial observation. *arXiv preprint arXiv:2406.17763* (2024).
43. Shysheya, A. *et al.* On conditional diffusion models for pde simulations. *Adv. Neural Inf. Process. Syst.* **37**, 23246–23300 (2024).
44. Rahman, M. A., Florez, M. A., Anandkumar, A., Ross, Z. E. & Azizzadenesheli, K. Generative adversarial neural operators. *arXiv preprint arXiv:2205.03017* (2022).
45. Oommen, V., Bora, A., Zhang, Z. & Karniadakis, G. E. Integrating neural operators with diffusion models improves spectral representation in turbulence modeling. *arXiv preprint arXiv:2409.08477* (2024).
46. Hu, P. *et al.* Wavelet diffusion neural operator. *arXiv preprint arXiv:2412.04833* (2024).

47. Shu, D., Li, Z. & Farimani, A. B. A physics-informed diffusion model for high-fidelity flow field reconstruction. *J. Comput. Phys.* **478**, 111972 (2023).
48. Qiu, J. *et al.* Pi-fusion: Physics-informed diffusion model for learning fluid dynamics. *arXiv preprint arXiv:2406.03711* (2024).
49. Bastek, J.-H., Sun, W. & Kochmann, D. M. Physics-informed diffusion models. *arXiv preprint arXiv:2403.14404* (2024).
50. Gao, H. *et al.* Bayesian conditional diffusion models for versatile spatiotemporal turbulence generation. *Comput. Methods Appl. Mech. Eng.* **427**, 117023 (2024).
51. Zhuang, Y., Cheng, S. & Duraisamy, K. Spatially-aware diffusion models with cross-attention for global field reconstruction with sparse observations. *Comput. Methods Appl. Mech. Eng.* **435**, 117623 (2025).
52. Jacobsen, C., Zhuang, Y. & Duraisamy, K. Cocogen: Physically consistent and conditioned score-based generative models for forward and inverse problems. *SIAM J. on Sci. Comput.* **47**, C399–C425 (2025).
53. Weed, J. & Bach, F. Sharp asymptotic and finite-sample rates of convergence of empirical measures in wasserstein distance. *Bernoulli* **25**, 2620–2648 (2019).
54. Dou, Z., Kotekal, S., Xu, Z. & Zhou, H. H. From optimal score matching to optimal sampling. *arXiv preprint arXiv:2409.07032* (2024).
55. Barron, A. R. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Inf. theory* **39**, 930–945 (1993).
56. Neyshabur, B., Tomioka, R. & Srebro, N. Norm-based capacity control in neural networks. In *Conference on Learning Theory*, 1376–1401 (2015).
57. Breiman, L. Hinging hyperplanes for regression, classification, and function approximation. *IEEE Transactions on Inf. Theory* **39**, 999–1013 (1993).
58. Klusowski, J. M. & Barron, A. R. Risk bounds for high-dimensional ridge function combinations including neural networks. *arXiv preprint arXiv:1607.01434* (2016).
59. Lu, L. *et al.* Physics-informed neural networks with hard constraints for inverse design. *SIAM J. on Sci. Comput.* **43**, B1105–B1132 (2021).
60. Ronneberger, O., Fischer, P. & Brox, T. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*, 234–241 (Springer, 2015).
61. Wang, S. *et al.* Cvit: Continuous vision transformer for operator learning. *arXiv preprint arXiv:2405.13998* (2024).
62. Hao, Z. *et al.* Dpot: Auto-regressive denoising operator transformer for large-scale pde pre-training. *arXiv preprint arXiv:2403.03542* (2024).
63. Ohana, R. *et al.* The well: a large-scale collection of diverse physics simulations for machine learning. *Adv. Neural Inf. Process. Syst.* **37**, 44989–45037 (2024).
64. McCabe, M. *et al.* Multiple physics pretraining for physical surrogate models. *arXiv preprint arXiv:2310.02994* (2023).
65. Chung, H., Kim, J., Mccann, M. T., Klasky, M. L. & Ye, J. C. Diffusion posterior sampling for general noisy inverse problems. In *The Eleventh International Conference on Learning Representations* (2023).
66. Lippmann, B. A. & Schwinger, J. Variational principles for scattering processes. I. *Phys. Rev.* **79**, 469–480 (1950).
67. Moulinec, H. & Suquet, P. A numerical method for computing the overall response of nonlinear composites with complex microstructure. *Comput. Methods Appl. Mech. Eng.* **157**, 69–94 (1998).
68. Gao, Y., Kolouri, S. & Duddu, R. Energy loss informed cell-based multilayer perceptron for solving elliptic pdes with multiscale features. *Available at SSRN 4997989* (2024).
69. Kou, C. *et al.* Efficient neural topology optimization via active learning for enhancing turbulent mass transfer in fluid channels. *arXiv preprint arXiv:2503.03997* (2025).
70. Vaswani, A. *et al.* Attention is all you need. *Adv. neural information processing systems* **30** (2017).
71. Xiong, R. *et al.* On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, 10524–10533 (PMLR, 2020).
72. Wang, S. *et al.* Bridging operator learning and conditioned neural fields: A unifying perspective. *arXiv preprint arXiv:2405.13998* (2024).

73. Tancik, M. *et al.* Fourier features let networks learn high frequency functions in low dimensional domains. *Adv. neural information processing systems* **33**, 7537–7547 (2020).
74. Qi, C. R., Su, H., Mo, K. & Guibas, L. J. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 652–660 (2017).
75. Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A. & Battaglia, P. Learning mesh-based simulation with graph networks. In *International conference on learning representations* (2020).
76. Peebles, W. & Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4195–4205 (2023).
77. Kingma, D. P. & Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
78. Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N. & Ganguli, S. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, 2256–2265 (PMLR, 2015).
79. Liu, Q. Rectified flow: A marginal preserving approach to optimal transport, 2022. URL <https://arxiv.org/abs/2209.14577> (2022).
80. Villani, C. *et al.* *Optimal transport: old and new*, vol. 338 (Springer, 2009).
81. Caffarelli, L. A. Boundary regularity of maps with convex potentials–ii. *Annals mathematics* **144**, 453–496 (1996).
82. Niles-Weed, J. & Berthet, Q. Minimax estimation of smooth densities in wasserstein distance. *The Annals Stat.* **50**, 1519–1540 (2022).
83. Woo, S. *et al.* Convnext v2: Co-designing and scaling convnets with masked autoencoders. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 16133–16142 (2023).
84. Wang, S., Wang, H. & Perdikaris, P. Learning the solution operator of parametric partial differential equations with physics-informed deepnets. *Sci. advances* **7**, eabi8605 (2021).
85. Driscoll, T. A., Hale, N. & Trefethen, L. N. *Chebfun guide* (2014).
86. Cox, S. M. & Matthews, P. C. Exponential time differencing for stiff systems. *J. Comput. Phys.* **176**, 430–455 (2002).
87. Wang, S., Liu, T.-R., Sankaran, S. & Perdikaris, P. Micrometer: Micromechanics transformer for predicting mechanical responses of heterogeneous materials. *arXiv preprint arXiv:2410.05281* (2024).

Acknowledgments

LL acknowledges support from the U.S. Department of Energy, Office of Advanced Scientific Computing Research, under Grants No. DE-SC0025593 and No. DE-SC0025592, and the U.S. National Science Foundation under Grants No. DMS-2347833 and No. DMS-2527294. SW acknowledges support from the NVIDIA Academic Grant Program. We thank Dr. Chenhui Kou for providing the turbulence mass transfer dataset and Dr. Mingyuan Zhou for insightful discussions.

Author contributions statement

S.W., Z.D. and L.L. conceived the project and developed the main methodology. S.W. implemented the core algorithms and conducted the experiments. Z.D. contributed to the theoretical analysis. T.L. and S.S. assisted with numerical simulation. L.L. supervised the project and provided critical revisions. All authors discussed the results and contributed to writing the manuscript.

S1 Notations

Our notations are summarized in Table S1.

Table S1. Main symbols and notations used in this work.

Notation	Description
FunDiff	
MLP	Multilayer perceptron
PE	Positional embedding
MSA	Multi-head self-attention
MHA	Multi-head attention
LN	Layer normalization
P	Patch size of Vision Transformer
D	Embedding dimension of Vision Transformer
Partial differential equations	
σ	Stress vector
$\mathbb{C}$	Fourth-order elastic tensor
$\mathbb{A}$	Strain concentration tensor
$E(x), \nu(x)$	A pair of Young's modulus and Poisson ratio
$\mathbf{u}$	Velocity field
w	Vorticity field
p	Pressure field
ν	Viscosity
Hyperparameters	
B	Batch size
Q	Number of query coordinates in each batch
D	Number of latent variables of interest
$H \times W$	Resolution of spatial discretization

We present detailed mathematical definitions as follows. The Wasserstein-1 distance (also known as the Earth Mover's Distance) between two probability measures μ and ν on a metric space $(\mathcal{X}, d)$ is defined as

$$W_1(\mu, \nu) = \inf_{\gamma \in \Pi(\mu, \nu)} \mathbb{E}_{(X, Y) \sim \gamma} [d(X, Y)],$$

where:

- $d(\cdot, \cdot)$ is the given metric (e.g., Euclidean distance) that measures the cost of transporting mass from X to Y .
- $\Pi(\mu, \nu)$ is the set of *couplings* of μ and ν , i.e., joint probability distributions γ on $\mathcal{X} \times \mathcal{X}$ whose marginals are μ and ν :

$$\gamma(A \times \mathcal{X}) = \mu(A), \quad \gamma(\mathcal{X} \times B) = \nu(B) \quad \text{for all measurable sets } A, B \subset \mathcal{X}.$$

Thus, $W_1(\mu, \nu)$ represents the minimal expected cost to transport the mass of μ to match ν , given that the cost function is the distance $d(x, y)$. In a dual form, W_1 distance can be equivalently expressed as

$$W_1(\mu, \nu) = \sup_{f \text{ is 1-Lipschitz}} \left[\int_{\mathcal{X}} f d\mu - \int_{\mathcal{X}} f d\nu \right].$$

Another notation we introduce here is the pushforward measure. Given two measurable spaces $(\mathcal{X}, \mathcal{A})$ and $(\mathcal{Y}, \mathcal{B})$, as well as a measurable function $T: \mathcal{X} \rightarrow \mathcal{Y}$, the pushforward measure $T_{\#}\mu$ of a measure μ on $(\mathcal{X}, \mathcal{A})$ is a measure on $(\mathcal{Y}, \mathcal{B})$ defined by

$$(T_{\#}\mu)(B) = \mu(T^{-1}(B)), \quad \forall B \in \mathcal{B}.$$

This means that for any measurable set $B \subset \mathcal{Y}$, the pushforward measure assigns a value based on how much mass μ assigns to its preimage $T^{-1}(B)$ in $\mathcal{X}$. In other words, $T_{\#}\mu$ is the underlying distribution of $T(x)$ where $x \sim \mu$.

Besides, for two functions $f(x), g(x) > 0$, we write $f \lesssim g$ if there exists a positive constant $C > 0$ such that $f(x) \leq C \cdot g(x)$ holds for sufficiently large $x > 0$. If $f \lesssim g$ and $g \lesssim f$ hold at the same time, then we denote it as $f \asymp g$, which means that there exist positive constants $c, C > 0$ such that $c \cdot g(x) \leq f(x) \leq C \cdot g(x)$ for sufficiently large $x > 0$.

S2 Preliminaries

Before introducing our generative models in function spaces, we provide some basic knowledge of diffusion models and its Minimax optimality.

We begin by reviewing foundational concepts of diffusion models in both discrete and continuous settings. Diffusion models form a class of generative frameworks that approximate complex data distributions through a two-phase process: a forward noising process that incrementally corrupts the data, followed by a backward denoising process that reconstructs data from noise.

In the discrete setting^{3,78}, the forward process is modeled as a first-order Markov chain with N steps:

$$q(\mathbf{x}_{1:N} | \mathbf{x}_0) = \prod_{k=1}^N q(\mathbf{x}_k | \mathbf{x}_{k-1}), \quad (\text{S1})$$

$$q(\mathbf{x}_k | \mathbf{x}_{k-1}) = \mathcal{N}(a_k \mathbf{x}_{k-1}, b_k^2 \mathbf{I}). \quad (\text{S2})$$

where $\mathbf{x}_0 \in \mathbb{R}^d$ is the original data point, $\mathbf{x}_k \in \mathbb{R}^d$ denotes the noised data at discrete time step k , and the coefficients $a_k \in \mathbb{R}$ and $b_k > 0$ are predefined scalars that control signal decay and noise level at step k . The forward process incrementally injects Gaussian noise while decaying the signal, and the Gaussian structure ensures that the marginal distribution $q(\mathbf{x}_k | \mathbf{x}_0)$ remains analytically tractable for all k . This facilitates both efficient sampling and likelihood estimation. During the backward process, a neural network $\mathbf{s}_\theta(\mathbf{x}_k, t_k)$ is trained to approximate the score function $\nabla_{\mathbf{x}_k} \log q(\mathbf{x}_k)$, where θ are the learnable network parameters and t_k is a scalar or embedding indicating the diffusion timestep associated with $\mathbf{x}_k$. The estimated score function is then used to reverse the noising process, enabling the generation of new samples from noise via stochastic denoising steps.

While classical diffusion models rely on stochastic differential equations (SDEs) or discrete Markov chains to reconstruct samples iteratively, rectified flow provides an appealing deterministic alternative. Instead of adding and removing noise through a stochastic process, rectified flow formulates the generative task as a deterministic transport problem, where a time-dependent vector field $\mathbf{v}(\mathbf{x}, t)$ is learned to smoothly morph a simple base distribution (e.g., standard Gaussian) into the data distribution⁷⁹. Specifically, rectified flow defines a continuous evolution via the ordinary differential equation (ODE):

$$\frac{d\mathbf{x}}{dt} = \mathbf{v}(\mathbf{x}, t),$$

where $\mathbf{x} \in \mathbb{R}^d$ is the evolving sample at time $t \in [0, 1]$ and $\mathbf{v} : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ is a learnable vector field optimized to carry mass from the source to the target distribution. In many cases, $\mathbf{v}(\mathbf{x}, t)$ is parameterized as the gradient of a scalar potential function, i.e., $\mathbf{v}(\mathbf{x}, t) = \nabla_{\mathbf{x}} \phi(\mathbf{x}, t)$, encouraging samples to travel along energy-efficient paths. This approach is rooted in the theory of optimal transport, aiming to minimize distortion and ensure that the learned trajectories follow nearly straight-line flows in high-dimensional space. The result is a computationally efficient and conceptually elegant framework for generative modeling.

To combine the advantages of deterministic flows and score-based models, recent work employs flow matching objectives. These objectives aim to learn a transport vector field by matching it to a known or estimated flow—such as the one induced by the diffusion model’s score function—via a regression loss. In this sense, flow matching acts as a unifying framework that bridges stochastic diffusion models and deterministic flow-based models. Importantly, rectified flow can be viewed as a special case of flow matching, where the target flow is derived from a score-based process and then refined into a smooth ODE trajectory. Notably, the c -rectified flow⁷⁹ extends this idea by introducing corrective updates to the vector field, refining the transport map through multiple iterations and improving convergence to the optimal transformation^{80,81}.

From this viewpoint, score-based diffusion models can be seen as stochastic analogs of deterministic flow models. Indeed, the backward process in a discrete diffusion model often takes the form

$$p_\theta(\mathbf{x}_{k-1} | \mathbf{x}_k) = \mathcal{N}(u_k \hat{\mathbf{x}}_0(\mathbf{x}_k) + v_k \mathbf{s}_\theta(\mathbf{x}_k, t_k), w_k^2 \mathbf{I}),$$

where $\hat{\mathbf{x}}_0(\mathbf{x}_k)$ is an estimate of the original data sample conditioned on $\mathbf{x}_k$, $u_k, v_k \in \mathbb{R}$ are scalar coefficients derived from the noise schedule, $w_k^2 > 0$ is the reverse variance at step k and $\mathbf{s}_\theta(\mathbf{x}_k, t_k)$ stands for the trained score function. As the variance w_k^2 approaches zero, this stochastic update converges to a deterministic transformation, recovering the structure of a flow model. This connection reveals that diffusion models, flow matching, and rectified flow all inhabit a shared conceptual space—each providing different approximations to the same underlying transport dynamics between probability distributions.

Unlike classical variance-preserving diffusion models³, which depend on carefully designed noise schedules for both forward and reverse processes, rectified flow directly parameterizes the dynamics through a deterministic ODE. This not only simplifies the learning pipeline but also improves inference efficiency by eliminating the need for repeated noise sampling. Despite its deterministic nature, rectified flow retains strong expressiveness and sample quality, and its connection to score matching and optimal transport principles makes it a compelling direction for scalable generative modeling.

Density estimation and the minimaxity of diffusion models

As a core problem in statistics and machine learning, the purpose of density estimation is to solve the following problem: given n i.i.d. data points $\{x_i\}_{i=1}^n$ sampled from an unknown distribution p^* , we want to construct an approximate distribution $\hat{p}$ based on the n given data points. Density estimation arises in diverse applications, from anomaly detection and clustering to modern generative modeling, where having an accurate handle on the underlying distribution is crucial. One broad class of solutions is parametric methods, which assume that the ground truth distribution p^* lies within a known family of distributions $\{p_\theta : \theta \in \Theta\}$. The density estimation problem then reduces to finding the optimal parametric vector $\hat{\theta}$ via log-likelihood maximization:

$$\hat{\theta} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \log p_\theta(x_i).$$

If p^* aligns well with the chosen family, parametric models can be computationally efficient and produce robust estimates. However, when data exhibit complex or multimodal patterns that deviate from the assumed structure, parametric methods can underfit or produce biased results. In contrast, non-parametric approaches place fewer constraints on the distribution's form. Kernel density estimation (KDE) is one of the most common examples:

$$\hat{p}_h(x) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{x-x_i}{h}\right).$$

Here, $K(\cdot)$ is a kernel function (which is usually chosen as Gaussian), h is the bandwidth, and d is the input dimension. Because nonparametric methods can model complex structures directly from data, they often uncover finer distributional details than simpler parametric models. However, as dimensionality increases, these methods face significant challenges, commonly referred to as the curse of dimensionality, leading to high sample complexity and potentially prohibitive computational costs.

Recently, diffusion models have emerged as powerful generative approaches that approximate the data distribution by reversing a forward noise-adding process. Although widely recognized for their empirical success, these models also admit a theoretical interpretation under the lens of minimax optimality for density estimation. In fact, under suitable assumptions on the function space that characterizes the ground truth distribution p^* , one can show that diffusion-based estimators achieve minimax rates for distribution (or score) recovery. Here we outline the theoretical underpinnings of this minimax perspective by first presenting a lower bound result (Theorem S1) that establishes a fundamental limit for density estimation in the space of all continuous functions $\mathcal{C}(\mathbb{R}^d)$, and then provide an upper bound result (Theorem S2) that shows how diffusion-based methods (and related estimators) can achieve this rate, thus elucidating the minimax guarantees behind modern generative modeling frameworks.

Theorem S1 (Lower Bound, Theorem 3 of Ref.⁸²). *Let $\alpha \geq 0$ denote the smoothness parameter, $d \geq 1$ the dimension, and $c > 0$ a positive constant. Suppose we consider the class of densities $\mathcal{F} \subseteq \mathcal{C}^\alpha(\mathbb{R}^d)$ satisfying*

- (i) *Smoothness: each density function $f \in \mathcal{F}$ has smoothness of order α .*
- (ii) *Lower bound: $f(x) \geq c$ for all $x \in \text{supp}(f)$, which means the density is lower bounded by a positive constant in its support set.*

Then the estimation rate in the Wasserstein-1 distance satisfies the following lower bound given n samples $\mu_1, \mu_2, \dots, \mu_n \sim f$:

$$\inf_{\hat{f}} \sup_{f \in \mathcal{F}} \mathbb{E}_f [W_1(\hat{f}, f)] \gtrsim \begin{cases} n^{-\frac{\alpha+1}{2\alpha+d}}, & d \geq 2, \\ n^{-1/2}, & d = 1, \end{cases}$$

where the infimum is taken over all estimators $\hat{f}$ based on the sample of size n .

Theorem S2 (Upper Bound, Section 3.3 of Ref.⁵⁴). *Let $\alpha \geq 0$ denote the smoothness parameter, $d \geq 1$ the dimension, and $c > 0$ a positive constant. We still consider the class of densities $\mathcal{F} \subseteq \mathcal{C}^\alpha(\mathbb{R}^d)$ defined in Theorem S1. By solving the backward diffusion SDE*

$$dY_t = \hat{s}(Y_t, T-t)dt + dW_t$$

with a non-parametric score estimator $\hat{s}(\cdot, t)$ injected, we achieve a density estimator $\hat{f}$ such that

$$\sup_{f \in \mathcal{F}} \mathbb{E}_f [W_1(\hat{f}, f)] \lesssim \begin{cases} n^{-\frac{\alpha+1}{2\alpha+d}}, & d > 2, \\ n^{-1/2} \log n, & d = 2, \\ n^{-1/2}, & d = 1. \end{cases}$$

From these two theorems, we can see that score-based diffusion models are able to achieve minimax rate over the Wasserstein distance with respect to density estimation.

S3 Theoretical proofs

In this section, we provide theoretical proofs of our theorems, lemmas, and propositions.

S3.1 Density estimation error decomposition

In this part, we prove the decomposition of density estimation Wasserstein error in (1). According to the FAE framework, we have $\hat{P} = G_{\#}\hat{p}$, which leads to

$$\begin{aligned} W_1(P, \hat{P}) &= W_1(P, G_{\#}\hat{p}) \stackrel{(a)}{\leq} W_1(P, G_{\#}p) + W_1(G_{\#}p, G_{\#}\hat{p}) \\ &\stackrel{(b)}{\leq} \mathbb{E}_{f \sim P} \|f - G \circ E(f)\|_2 + \text{Lip}(G) \cdot W_1(p, \hat{p}). \end{aligned}$$

Here, (a) is the direct result of the triangle inequality. (b) holds because of the following two inequalities:

$$\begin{aligned} W_1(P, G_{\#}p) &= W_1(P, G_{\#}E_{\#}P) = \sup_{\text{Lip}(\mathcal{L}) \leq 1} [\mathbb{E}_{f \sim P} \mathcal{L}(f) - \mathbb{E}_{f \sim P} \mathcal{L}(G \circ E(f))] \leq \mathbb{E}_{f \sim P} \|f - G \circ E(f)\|_2, \\ W_1(G_{\#}p, G_{\#}\hat{p}) &\leq \sup_{\theta, \theta'} \frac{\|G(\theta) - G(\theta')\|_2}{\|\theta - \theta'\|_2} \cdot W_1(p, \hat{p}) = \text{Lip}(G) \cdot W_1(p, \hat{p}). \end{aligned}$$

Then it comes to our conclusion.

S3.2 Proof of Theorem 1

For any function $f \in \mathcal{B}$, denote its eigen-decomposition as $f = \sum_{i=1}^{\infty} \theta_i \cdot \sqrt{\mu_i} \psi_i$, then the reconstructed function

$$G \circ E(f) = \sum_{i=1}^D \theta_i \cdot \sqrt{\mu_i} \psi_i.$$

The reconstruction loss for $f \in \mathcal{B}$

$$\|f - G \circ E(f)\|_2^2 = \sum_{i=1}^D \theta_i^2 \cdot \mu_i \leq \mu_D \cdot \sum_{i=1}^D \theta_i^2 \leq \mu_D \cdot \sum_{i=1}^{\infty} \theta_i^2 = \mu_D \cdot \|f\|_{\mathcal{H}}^2 \leq \mu_D,$$

which leads to

$$\mathbb{E}_{f \sim P} \|f - G \circ E(f)\|_2 \leq \sqrt{\mu_D}.$$

For the next term $\text{Lip}(G)$, we consider two vectors $\theta, \theta' \in \mathbb{R}^D$. Denote $f, f' \in \mathcal{B}$ as $f = G(\theta) = \sum_{i=1}^D \theta_i \cdot \sqrt{\mu_i} \psi_i$ and $f' = G(\theta') = \sum_{i=1}^D \theta'_i \cdot \sqrt{\mu_i} \psi_i$, then

$$\|f - f'\|_2^2 = \sum_{i=1}^D (\theta_i - \theta'_i)^2 \cdot \mu_i \leq \mu_1 \cdot \|\theta - \theta'\|_2^2,$$

and we obtain that $\text{Lip}(G) = \sqrt{\mu_1}$. Given the oracle loss $W_1(p, \hat{p}) \lesssim \varepsilon(D, n) = n^{-1/D}$, we have

$$W_1(P, \hat{P}) \leq \sqrt{\mu_D} + \sqrt{\mu_1} \cdot n^{-1/D}.$$

Under the polynomial decay setting:

$$W_1(P, \hat{P}) \lesssim n^{-1/D} + D^{-\beta}.$$

When the latent dimension is chosen as $D = \frac{1}{\beta} \cdot \frac{\log n}{\log \log n}$, we obtain that

$$W_1(P, \hat{P}) \lesssim \left(\frac{\log \log n}{\log n} \right)^{\beta}.$$

Under the exponential decay setting:

$$W_1(P, \hat{P}) \lesssim n^{-1/D} + \exp(-C_1/2 \cdot D^{\gamma}).$$

When we choose the optimal dimension $D = \left(\frac{2 \log n}{C_1} \right)^{\frac{1}{1+\gamma}}$, we obtain that

$$W_1(P, \hat{P}) \lesssim \exp \left(-(C_1/2)^{\frac{1}{1+\gamma}} (\log n)^{\frac{\gamma}{1+\gamma}} \right).$$

S3.3 Proof of Theorem 5

In this proof, we still use the density loss decomposition. For the representation loss, we already have

$$\|f - G \circ E(f)\|_2 = \|f - f(\tilde{\theta}; \mathbf{x})\|_2 \leq \sqrt{\mathbb{E}_{\mathbf{x} \sim \text{Unif}(\Omega)} |f(\mathbf{x}) - f(\tilde{\theta}; \mathbf{x})|^2} \leq \frac{4}{\sqrt{m}}$$

holds for $\forall f \in \mathcal{F}_s$ according to the approximation error in Lemma 4, which leads to

$$\mathbb{E}_{f \sim P} \|f - G \circ E(f)\|_2 \leq \frac{4}{\sqrt{m}}.$$

For the next term $\text{Lip}(G)$, we can prove the following result.

Lemma S3. For any two parameter sets $(a_k, \mathbf{b}_k, c_k)_{k \in [m]}$ and $(a'_k, \mathbf{b}'_k, c'_k)_{k \in [m]}$ where $|a_k|, |a'_k| \leq \frac{2}{m}$, $\|\mathbf{b}_k\|_1 = \|\mathbf{b}'_k\|_1 = 1$ and $|c_k|, |c'_k| \leq 1$ for $\forall k \in [m]$, then for $\theta = (\frac{m}{2}a_k, \mathbf{b}_k, c_k)$ and $\theta' = (\frac{m}{2}a'_k, \mathbf{b}'_k, c'_k)$, it holds that

$$\|f(\theta; \cdot) - f(\theta'; \cdot)\|_2 \lesssim \sqrt{\frac{d}{m}} \cdot \|\theta - \theta'\|_2.$$

Notice that we conduct a normalization on $\{a_k\}$ in the encoder and the decoder.

Proof of Lemma S3. Notice that for $\theta = (\frac{m}{2}a_k, \mathbf{b}_k, c_k)$ and $\theta' = (\frac{m}{2}u_k, \mathbf{v}_k, w_k)$, we have

$$\begin{aligned} |f(\theta; \mathbf{x}) - f(\theta'; \mathbf{x})| &= \left| \sum_{k=1}^m a_k \sigma(\mathbf{b}_k \cdot \mathbf{x} + c_k) - \sum_{k=1}^m u_k \sigma(\mathbf{v}_k \cdot \mathbf{x} + w_k) \right| \\ &\leq \left| \sum_{k=1}^m (a_k - u_k) \cdot \sigma(\mathbf{b}_k \cdot \mathbf{x} + c_k) + \sum_{k=1}^m u_k \cdot (\sigma(\mathbf{b}_k \cdot \mathbf{x} + c_k) - \sigma(\mathbf{v}_k \cdot \mathbf{x} + w_k)) \right| \\ &\leq 2\|a - u\|_1 + \frac{2}{m} \cdot \sum_{k=1}^m (|\mathbf{b}_k - \mathbf{v}_k| \cdot |\mathbf{x}| + |c_k - w_k|) \\ &\leq \frac{4}{m} \left\| \frac{m}{2}a - \frac{m}{2}u \right\|_1 + \frac{2}{m} \sum_{k=1}^m \|\mathbf{b}_k - \mathbf{v}_k\|_1 + \frac{2}{m} \|c - w\|_1 \\ &\leq \frac{4}{\sqrt{m}} \left\| \frac{m}{2}a - \frac{m}{2}u \right\|_2 + \frac{2\sqrt{d}}{m} \sum_{k=1}^m \|\mathbf{b}_k - \mathbf{v}_k\|_2 + \frac{2}{\sqrt{m}} \|c - w\|_2 \\ &\leq \left(\frac{16}{m} + m \cdot \frac{4d}{m^2} + \frac{4}{m} \right)^{1/2} \cdot \left(\left\| \frac{m}{2}a - \frac{m}{2}u \right\|_2^2 + \sum_{k=1}^m \|\mathbf{b}_k - \mathbf{v}_k\|_2^2 + \|c - w\|_2^2 \right)^{1/2} \\ &= \frac{\sqrt{20+4d}}{\sqrt{m}} \cdot \|\theta - \theta'\|_2 \lesssim \sqrt{\frac{d}{m}} \cdot \|\theta - \theta'\|_2. \end{aligned}$$

The inequality above holds for $\forall \mathbf{x} \in \Omega = [0, 1]^d$, which leads to

$$\|f(\theta; \cdot) - f(\theta'; \cdot)\|_2 \leq \|f(\theta; \cdot) - f(\theta'; \cdot)\|_\infty \lesssim \sqrt{\frac{d}{m}} \cdot \|\theta - \theta'\|_2,$$

and it comes to our conclusion. $\square$

This lemma concludes that $\text{Lip}(G) \lesssim \sqrt{d/m}$ in (1). Finally, the latent dimension $D = m(2+d)$ and the oracle loss

$$W_1(p, \hat{p}) \lesssim \varepsilon(D, n) = n^{-1/D} = \exp\left(-\frac{\log n}{m(2+d)}\right).$$

To sum up, it holds that

$$W_1(P, \hat{P}) \lesssim \frac{4}{\sqrt{m}} + \frac{\sqrt{d}}{\sqrt{m}} \exp\left(-\frac{\log n}{m(2+d)}\right).$$

When choosing the optimal network width $m = \frac{2 \log n}{(2+d) \log d}$, we obtain the optimal rate:

$$W_1(P, \hat{P}) \lesssim \sqrt{\frac{d \log d}{\log n}}.$$

S3.4 Lower bound: Proof of Theorem 6

Step 1: Bi-Lipschitz embedding of a Euclidean ball into RKHS Ball $\mathcal{B}$

For a fixed dimension $d \geq 1$, define the following linear map $T_d : \mathbb{R}^d \rightarrow \mathcal{H}$ by

$$S_d(t) := \sum_{i=1}^d \sqrt{\mu_i} \cdot t_i \phi_i,$$

where $\{\mu_i\}$ and $\{\phi_i\}$ are the eigenvalues and eigen-functions of RKHS $\mathcal{H}$, respectively. For vector $t \in \mathbb{R}^d$, we have

$$\|T_d(t)\|_{\mathcal{H}}^2 = \sum_{i=1}^d \frac{(\sqrt{\mu_i} \cdot t_i)^2}{\mu_i} = \sum_{i=1}^d t_i^2 = \|t\|_2^2.$$

Hence $T_d : t \in B_d(1) \mapsto T_d(t) \in \mathcal{B}$, where $B_d(1) := \{t \in \mathbb{R}^d : \|t\|_2 \leq 1\}$ is a d -dimensional Euclidean ball. Moreover, for all $s, t \in \mathbb{R}^d$, we have

$$\|T_d(t) - T_d(s)\|_{L_2(\nu)}^2 = \sum_{i=1}^d \mu_i (t_i - s_i)^2 \Rightarrow \sqrt{\min_{i \in [d]} \mu_i} \cdot \|t - s\|_2 \leq \|T_d(t) - T_d(s)\|_{L_2(\nu)} \leq \sqrt{\max_{i \in [d]} \mu_i} \cdot \|t - s\|_2.$$

Since $\mu_1 \asymp 1, \mu_d \asymp d^{-2\beta}$, the lower Lipschitz factor is

$$L_d := \sqrt{\mu_d} \geq \sqrt{c} \cdot d^{-\beta}.$$

Now, we introduce the pushforward class. Let $\mathcal{Q}_d$ be the set of probability measures supported on the d -dimensional unit Euclidean ball $B_d(1)$. Consider its image

$$\mathcal{P}_d := \{T_d \# Q : Q \in \mathcal{Q}_d\} \subseteq \{P : \text{supp}(P) \subseteq \mathcal{B}\}.$$

Step 2: Transferring Wasserstein distance through the embedding

For any $Q, Q' \in \mathcal{Q}_d$, by the definition of W_1 and the lower Lipschitz bound,

$$W_1(T_d \# Q, T_d \# Q') \geq L_d \cdot W_1^{\mathbb{R}^d}(Q, Q'),$$

where $W_1^{\mathbb{R}^d}$ is the Wasserstein-1 metric in $\mathbb{R}^d$ with Euclidean ground norm. Therefore, we can apply estimator pullback technique and use the statistical lower bound for density estimation inside d -dimensional unit ball (Euclidean space) to cap the statistical lower bound for density estimation inside an RKHS ball (function space).

Given any estimator $\hat{P}_n = \hat{P}_n(X_{1:n})$ taking values in probability measures on $\mathcal{B}$, define a corresponding $\mathbb{R}^d$ -estimator $\hat{Q}_n$ on data $Y_i \in B_d(1)$ by

$$\hat{Q}_n(Y_{1:n}) \in \arg \min_{Q \in \mathcal{Q}_d} W_1(T_d \# Q, \hat{P}_n(T_d(Y_1), T_d(Y_2), \dots, T_d(Y_n))).$$

By the optimality of the argmin and the inequality above, we can conclude that

$$W_1(T_d \# Q, \hat{P}_n(T_d(Y_{1:n}))) \geq L_d \cdot W_1^{\mathbb{R}^d}(Q, \hat{Q}_n(Y_{1:n})).$$

Taking expectation under $Y_i \sim Q$ and taking supremum over $Q \in \mathcal{Q}_d$, we obtain the risk domination

$$\sup_{P \in \mathcal{P}_d} \mathbb{E}[W_1(P, \hat{P}_n)] \geq L_d \cdot \sup_{Q \in \mathcal{Q}_d} \mathbb{E}[W_1^{\mathbb{R}^d}(Q, \hat{Q}_n)].$$

Finally, we take the infimum over all original estimators $\hat{P}_n$ (which induces taking infimum over all such $\hat{Q}_n$) yields

$$\inf_{\hat{P}_n} \sup_{P \in \mathcal{P}_d} \mathbb{E}[W_1(P, \hat{P}_n)] \geq L_d \cdot \left(\inf_{\hat{Q}_n} \sup_{Q \in \mathcal{Q}_d} \mathbb{E}[W_1^{\mathbb{R}^d}(Q, \hat{Q}_n)] \right) := R_n^{(d)}.$$

Step 3: A d -dimensional Wasserstein lower bound on the Euclidean ball

For each fixed $d \geq 1$, a standard conclusion of the classical curse-of-dimensionality for Wasserstein estimation tells us that, on a bounded set of $\mathbb{R}^d$, the minimax W_1 risk scales as $R_n^{(d)} \gtrsim n^{-1/d}$. Therefore,

$$\inf_{\hat{P}_n} \sup_{P \in \mathcal{P}_d} \mathbb{E}[W_1(P, \hat{P}_n)] \geq L_d \cdot R_n^{(d)} \gtrsim d^{-\beta} \cdot n^{-1/d}.$$

Step 4: Optimize the bound over dimension d

For any $d \geq 1$, since $\mathcal{P}_d \subset \{P : \text{supp}(P) \subseteq \mathcal{B}\}$, we have

$$\mathcal{R}_n := \inf_{\hat{P}_n} \sup_{P: \text{supp}(P) \subseteq \mathcal{B}} \geq \inf_{\hat{P}_n} \sup_{P \in \mathcal{P}_d} \mathbb{E} \left[W_1(P, \hat{P}_n) \right] \gtrsim d^{-\beta} \cdot n^{-1/d}.$$

Now we choose an optimal dimension d to maximize $d^{-\beta} \cdot n^{-1/d}$. By basic calculus, it is obvious that the maximizer $d^* = \frac{\log n}{\beta}$, where $d^{-\beta} \cdot n^{-1/d} = (\beta/e)^\beta \cdot \left(\frac{1}{\log n} \right)^\beta$. To sum up, we have

$$\mathcal{R}_n := \inf_{\hat{P}_n} \sup_{P: \text{supp}(P) \subseteq \mathcal{B}} \gtrsim \left(\frac{1}{\log n} \right)^\beta,$$

which comes to our conclusion.

S4 Experimental details

S4.1 Baselines

In our experiments, all baseline models are trained using the same procedure described in the Results section. Specifically, each model is trained for 10^5 iterations using the AdamW optimizer with a weight decay of 10^{-5} and a batch size of 16. The learning rate schedule includes a warm-up of 2,000 steps from zero to the base learning rate. For neural operator baselines, we use a base learning rate of 10^{-3} with exponential decay every 2,000 steps at a rate of 0.9. For diffusion-based models, we use a base learning rate of 10^{-4} with the same decay schedule, and apply exponential moving average (EMA) updates to stabilize training. For randomly downsampled inputs, we interpolate them back to the original resolution before processing.

Fourier Neural Operator (FNO). FNO architecture leverages Fourier transform to efficiently learn mappings between function spaces, making it particularly effective for modeling PDEs and physical systems. We employ an FNO model with four layers, where the number of channels is 32, and the Fourier modes is 32. The GeLU activation function is employed, and no normalization scheme is applied.

Vision Transformer (ViT). ViT extends the transformer architecture to visual tasks by utilizing self-attention mechanisms that capture global dependencies across input fields, offering superior performance on a range of spatial prediction problems. We adopt a ViT architecture that processes inputs by splitting them into patches of size 16. The network consists of 8 transformer blocks, each with an embedding dimension of 256, an MLP ratio of 2, and 8 attention heads.

UNet. UNet implements a symmetric encoder-decoder framework with skip connections that preserves multi-resolution features, enabling effective semantic segmentation and dense prediction tasks. We implement a conventional UNet architecture⁶⁰ with group normalization. The model uses an embedding dimension of 64 for its convolutional layers. We use GeLU activations and group normalization.

Continuous Vision Transformer (CViT)⁶¹. A neural field architecture that integrates a ViT encoder with trainable grid and coordinate embeddings for continuous, resolution-agnostic querying. In our implementation, the encoder operates on 16×16 patches with an embedding dimension of 256, depth 8, and 8 attention heads (MLP ratio 2). The decoder adopts a similar configuration with an embedding dimension of 256, depth 4, and 8 attention heads, followed by two MLP layers to obtain the final output.

CNextU-Net⁶³. A modernized U-Net backbone built upon ConvNeXt blocks⁸³, providing a high-capacity convolutional baseline competitive with transformer-based architectures for spatiotemporal field regression. Our model employs a four-stage encoder-decoder hierarchy with two ConvNeXt blocks per stage and two additional blocks at the bottleneck, initialized with 32 feature channels and doubling features at each downsampling level.

AViT⁶⁴. An attention-based ViT variant that sequentially attends over spatial and temporal dimensions, enabling efficient modeling of dynamic physical systems. Our implementation follows the base model described in Ref.⁶⁴, consisting of 8 AViT blocks with hidden dimension 512, 8 attention heads, and grouped axial attention across spatial partitions.

DPOT⁶². A PDE foundation operator trained via autoregressive denoising and Fourier attention, designed to generalize across diverse PDE systems and boundary conditions. In our implementation, DPOT processes inputs of size 256×256 with a patch size of 16, embedding dimension 512 and an MLP ratio of 2. The core architecture consists of a Fourier-attention transformer stack with 8 layers, interleaved with 4 Fourier mixer blocks (via AFNO) for spatial-frequency mixing.

Diffusion Posterior Sampling (DPS)⁶⁵. An inverse solver for general operators $\mathcal{A}$ that augments the reverse diffusion process with a likelihood-score guidance term $\nabla_x \log p_\sigma(y | \mathcal{A}(x))$ to approximate posterior sampling. We adopt the original denoising diffusion probabilistic model (DDPM) backbone with four encoder blocks and feature depths (32, 64, 128, 256), each comprising two residual blocks and group normalization (8 groups). Attention is applied in the last two stages with eight heads, and the model uses an embedding width of 256 and one middle residual block. Sampling is performed over 1,000 diffusion steps using a cosine noise schedule and a tuned guidance scale of $\lambda = 1000$, selected for best reconstruction accuracy.

DiffusionPDE⁴². A conditional diffusion model that jointly learns the distribution of PDE coefficients and solutions, guided by both observation consistency and PDE residual constraints during denoising. We follow the official implementation with a four-stage U-Net backbone using feature channels (64, 128, 256, 512), two residual blocks per stage, and self-attention in the bottleneck layer. Training is conducted for 1,000 diffusion steps with a cosine noise schedule and dual guidance weights tuned to balance data fidelity and physics consistency, selected for optimal reconstruction accuracy.

Cross-Attention Diffusion (Cross-Attn)⁵¹. A diffusion framework that employs cross-attention to reconstruct spatially distributed physical fields from sparse or irregular measurements. Following the original implementation, we use the aforementioned U-Net as the backbone. For the conditioning, measurement tokens are obtained through patch embeddings and processed by 4 transformer layers with an embedding dimension of 256, MLP ratio 2, and 8 attention heads. Cross-attention modules are integrated into each U-Net residual block to encode measurement-conditioned context and guide the denoising process.

S4.2 Kolmogorov flow

Data generation. We utilize the dataset generated by Li et al.¹³, solved using a pseudo-spectral solver. The initial condition $\omega_0(x)$ is sampled from a Gaussian random field, $\mathcal{N}(0, 73/2(-\Delta + 49I)^{-5/2})$. The simulation employs a 2048×2048 uniform grid, with 40 sequences spanning 10 seconds each ($T = 10$). The data is downsampled to a 256×256 spatial grid with a fixed temporal resolution of $\Delta t = 1/32$ seconds, yielding a dataset of 40 sequences, each containing 320 frames.

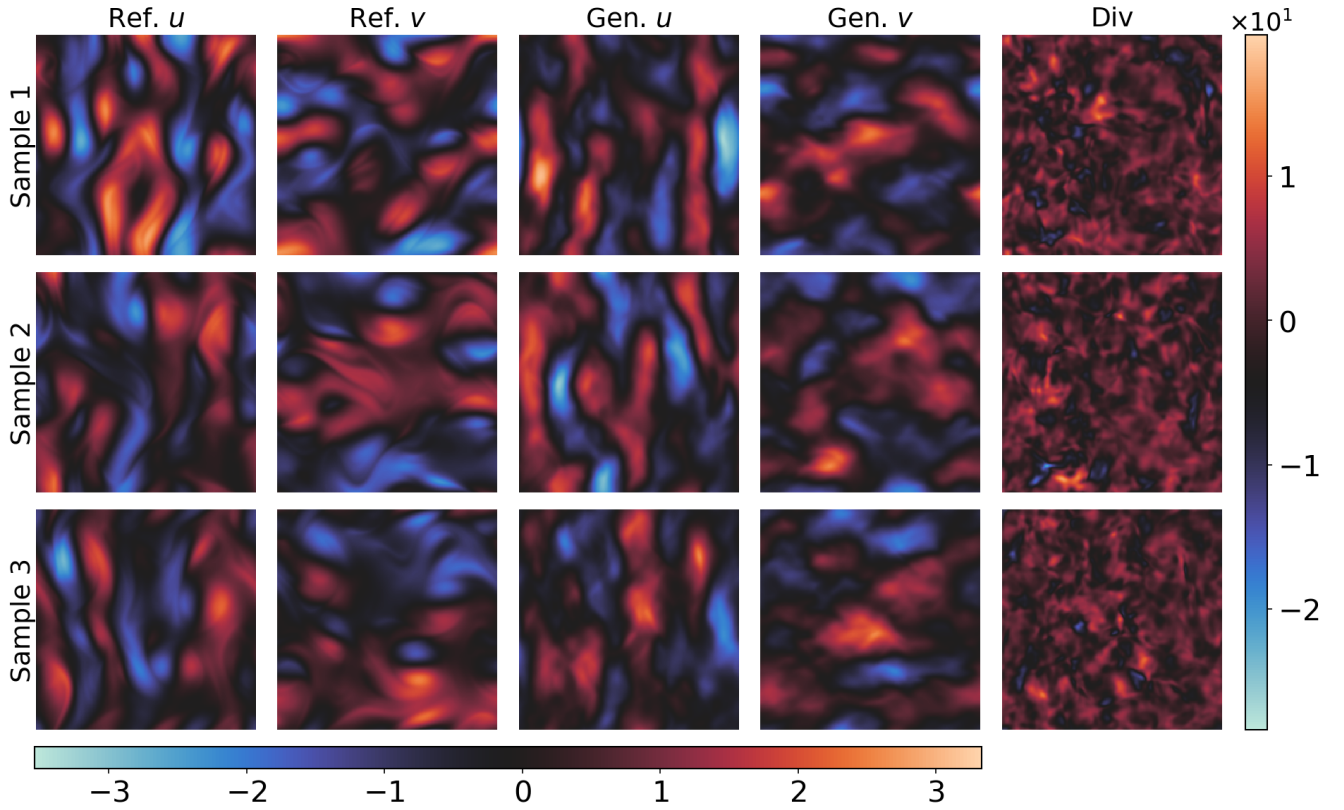


Figure S1. Kolmogorov flow. Unconditional generation without enforced divergence-free constraint. Examples of three generated velocity fields and their corresponding divergence fields.

S4.3 Burgers' equation

Data generation. Following Wang et al.⁸⁴, we generate our dataset by sampling 4,000 initial conditions from a GRF with covariance operator $\mathcal{N}(0, 25^2(-\Delta + 5^2I)^{-4})$. For each initial condition, we solve the Burgers' equation in (5) using spectral methods with periodic boundary conditions. The numerical solution is computed using the Chebfun package⁸⁵ with a spectral Fourier discretization and the fourth-order ETDRK4 time-stepping scheme⁸⁶, using a time-step size of 10^{-4} . We record solution snapshots every $\Delta t = 0.005$ up to $t = 1$, resulting in 201 temporal snapshots for each realization on a uniform 200×201 spatio-temporal grid.

S4.4 Linear elasticity

Data generation. The RVE domain $\Omega = [0, 1]^2$ is considered as a fiber-reinforced composite microstructure, where circular fibers are embedded within the matrix in a hexagonal lattice arrangement. The material properties including $E(\mathbf{x})$ and $\nu(\mathbf{x})$ are

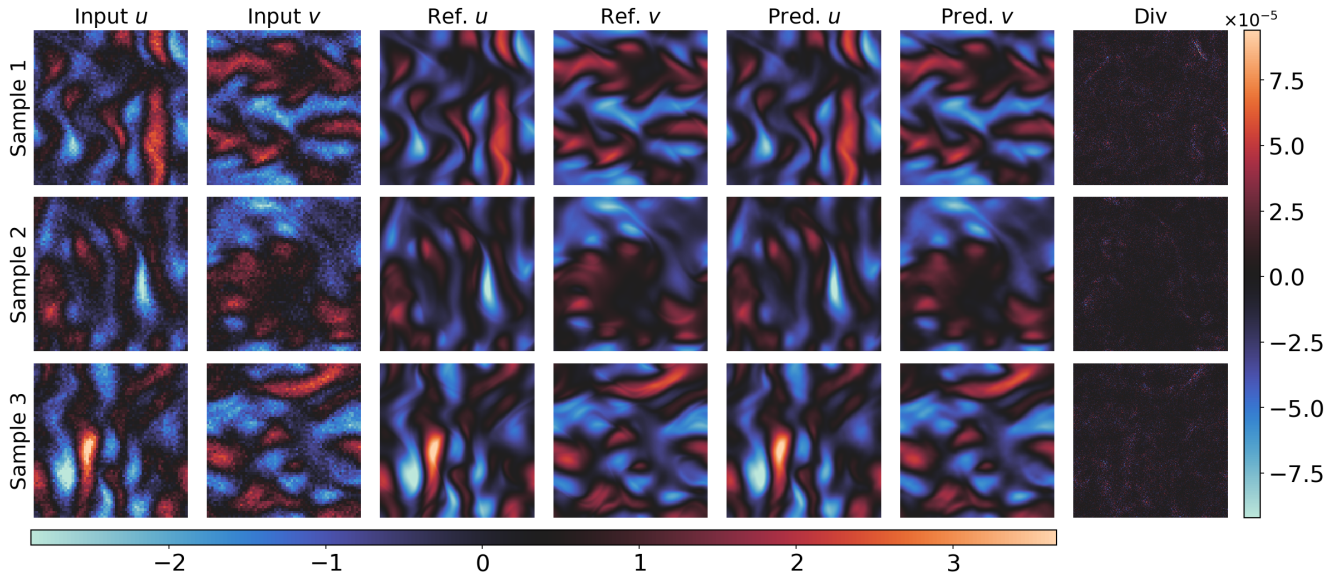


Figure S2. Kolmogorov flow. Examples of the model predictions for three sample solutions. Each row shows a different sample, displaying from left to right: downsampled input velocity components (u , v), reference high-resolution velocity fields, predicted velocity fields, and the predicted divergence field. The model successfully reconstructs sharp flow features from the coarse input while strictly maintaining the divergence-free condition, as evidenced by the negligible values in the divergence fields (rightmost column).

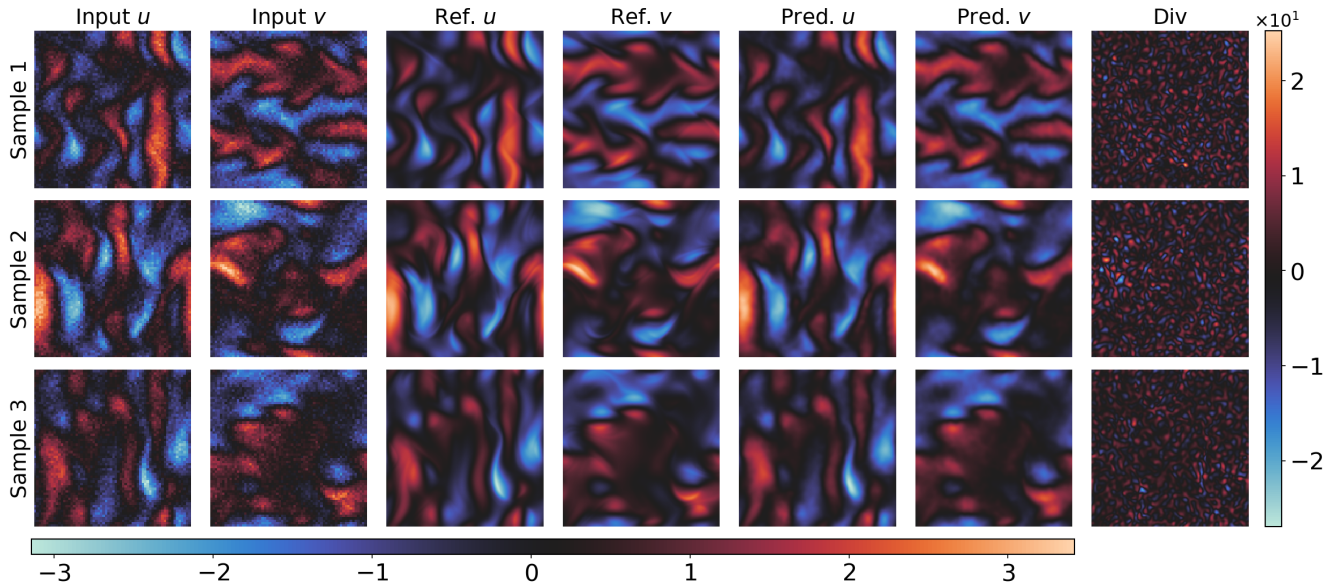


Figure S3. Kolmogorov flow. Examples of the model predictions for three sample solutions. Each row shows a different sample with: downsampled input velocity components (u , v), reference high-resolution fields, predicted velocity fields, and the predicted divergence. While the model achieves basic upsampling of the velocity fields, the reconstructions exhibit noticeable blurring compared to the sharp features in the reference solutions. The large values in the divergence fields (rightmost column) indicate that the incompressibility constraint is not enforced.

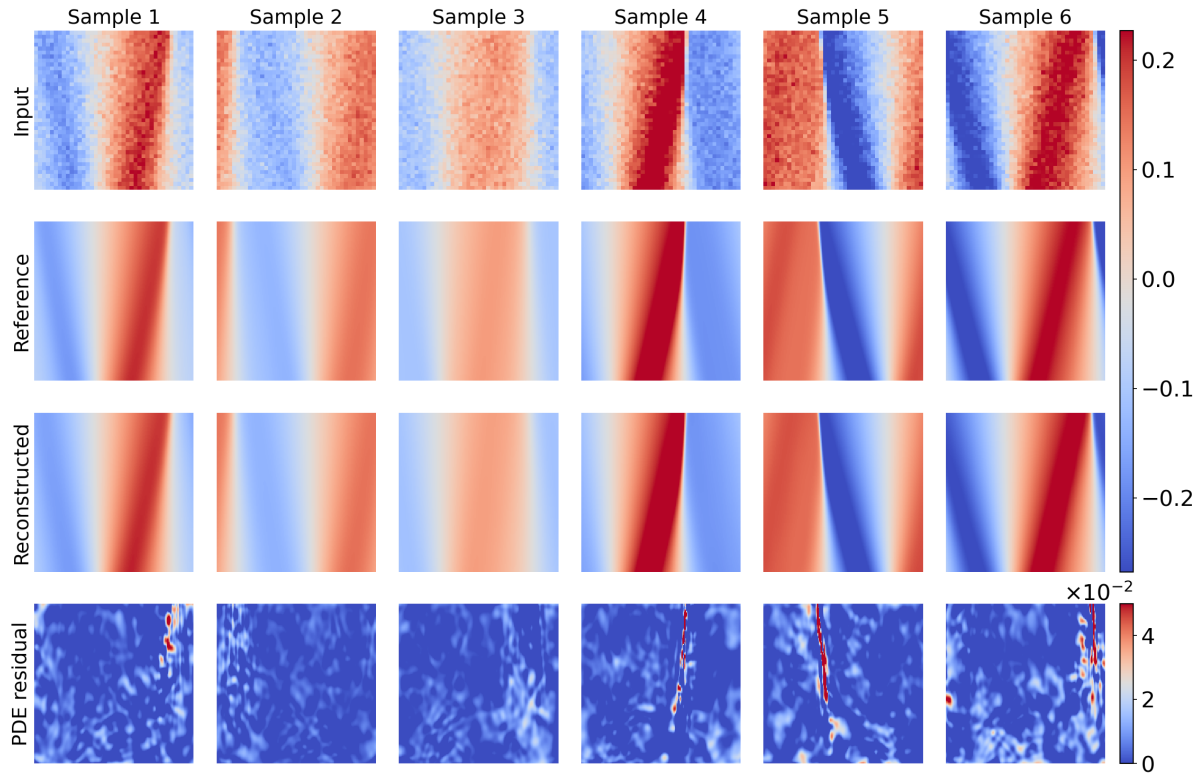


Figure S4. Burgers' equation. Representative generated PDE solutions and the corresponding PDE residuals, given the coarse measurements in 50×50 space-time resolution. The model is trained with incorporated PDE residual constraints.

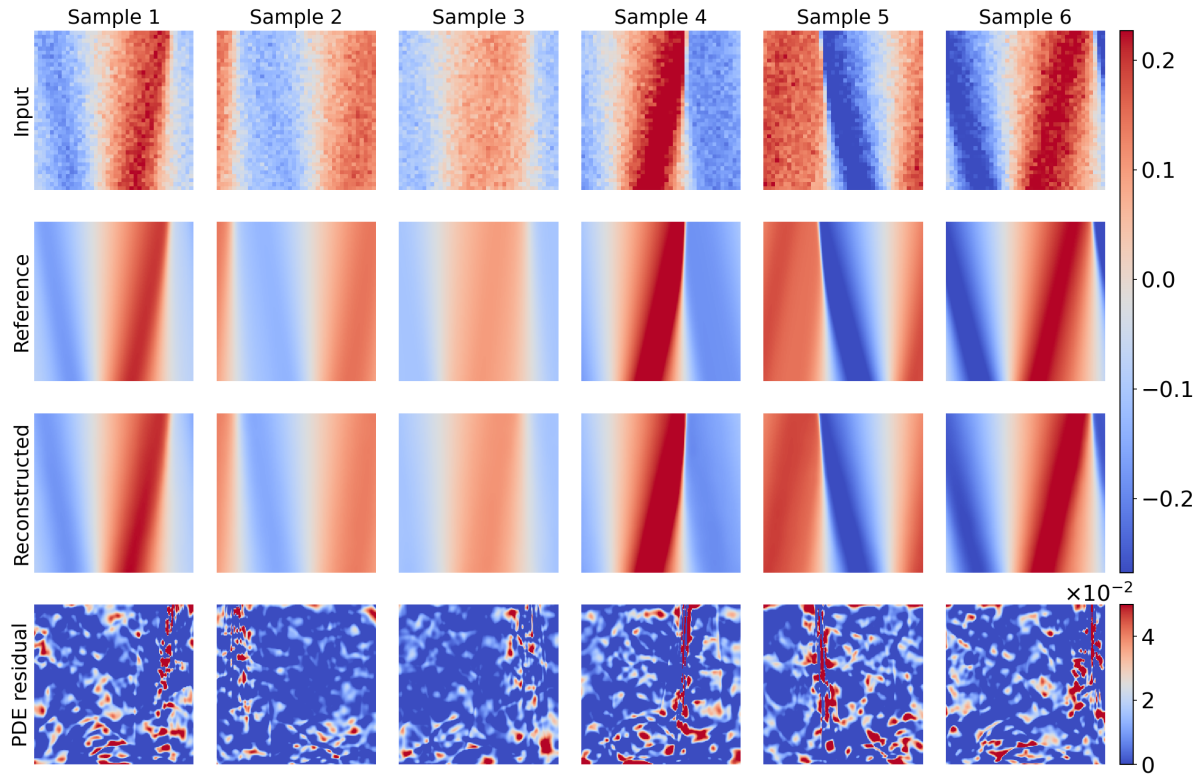


Figure S5. Burgers' equation. Representative generated PDE solutions and the corresponding PDE residuals, given the coarse measurements in 50×50 space-time resolution. The model is trained without incorporating PDE residual constraints.

homogeneous inside the disjointed matrix or fiber phases:

$$E(\mathbf{x}), \nu(\mathbf{x}) = \begin{cases} E_f, \nu_f, & \text{if } \mathbf{x} \in \Omega_f, \\ E_m, \nu_m, & \text{if } \mathbf{x} \in \Omega_m, \end{cases}$$

where Ω_f and Ω_m represent fiber and matrix phases, respectively. Our dataset comprises 7,500 unique RVEs, each discretized on a uniform 256×256 grid. For all RVEs, the matrix material properties are fixed with $E_m = 1$ (GPa) and $\nu_m = \nu_f = 0.35$. The volume fraction ν_f and the Young's modulus E_f of fiber are randomly generated through the Latin hypercube sampling (LHS) design method, ensuring that $\nu_f \in [0.1, 0.65]$ and $E_f \in [10^{-6}, 10^6]$. The $A_{ijkl}(\mathbf{x})$ of each RVE is computed by solving (7) subjected to (8) using a Fast Fourier Transform (FFT)-based homogenization method⁶⁷. A detailed and comprehensive implementation of generating $A_{ijkl}(\mathbf{x})$ can be found in Ref.⁸⁷. Given that the diagonal components A_{11} , A_{22} , and A_{33} of the strain concentration tensor dominate the mechanical response, these components are exclusively used to train our functional autoencoder.

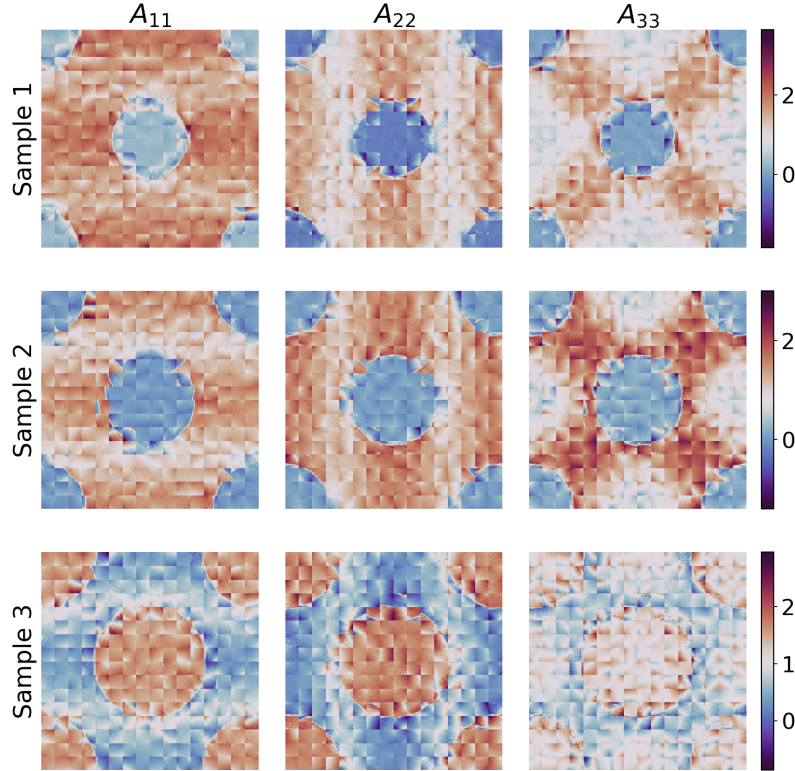


Figure S6. Linear elasticity. Generative samples obtained by training a discrete ViT autoencoder and latent diffusion model using the pipeline in Fig. 1. Noticeable discontinuities appear in the generated outputs, manifesting as distinct patch artifacts.

S4.5 Turbulence mass transfer

Data generation. Various topologies are randomly generated and evenly assigned them to 9 different inlet velocities ($v \in \{0.1, 0.2, \dots, 0.9\}$ m/s). The mechanistic model is used to obtain the concentration and pressure distributions, corresponding to the given v and $\gamma(x, y)$. To facilitate the training of neural networks, the total pressure p , including static pressure and dynamic pressure, is scaled to a uniform range, where $P = \frac{p}{2000} + 0.26$. To ensure the effectiveness of the generated topology, the following solid volume constraint is set:

$$\iint (1 - \gamma) dV \geq \alpha V_\Omega,$$

where $\alpha = 0.1$ is the minimum solid volume fraction in the design domain, and V_Ω is the volume of the design domain. We have 900 samples for training and 55 for testing.